

DTS103TC

Data Structure and Algorithms

Lecture 3: Data Structures 1

Dr. Qi Chen
School of AI and Advanced Computing

Learning outcomes

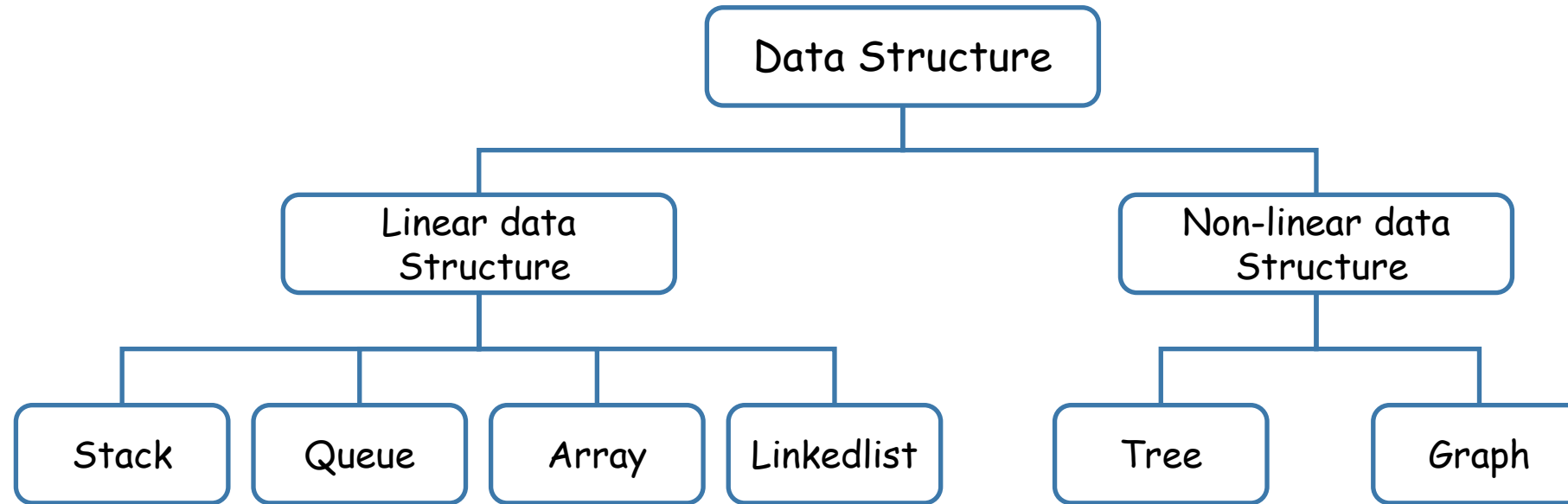
- Stack
- Queue
- LinkedList
 - Doubly Linked List
- Array
- Tree
 - Binary Tree
- Heap
 - Heapsort

What is a Data Structure?

- A data structure is a way of **organizing and storing data** so that it can be **accessed and used efficiently**.
 - Imagine you have **1000 files** on your computer. How can you organize them?
 - Put all files on the desktop
 - Organize files into folders and subfolders
- Different ways of organizing data lead to different efficiencies.**

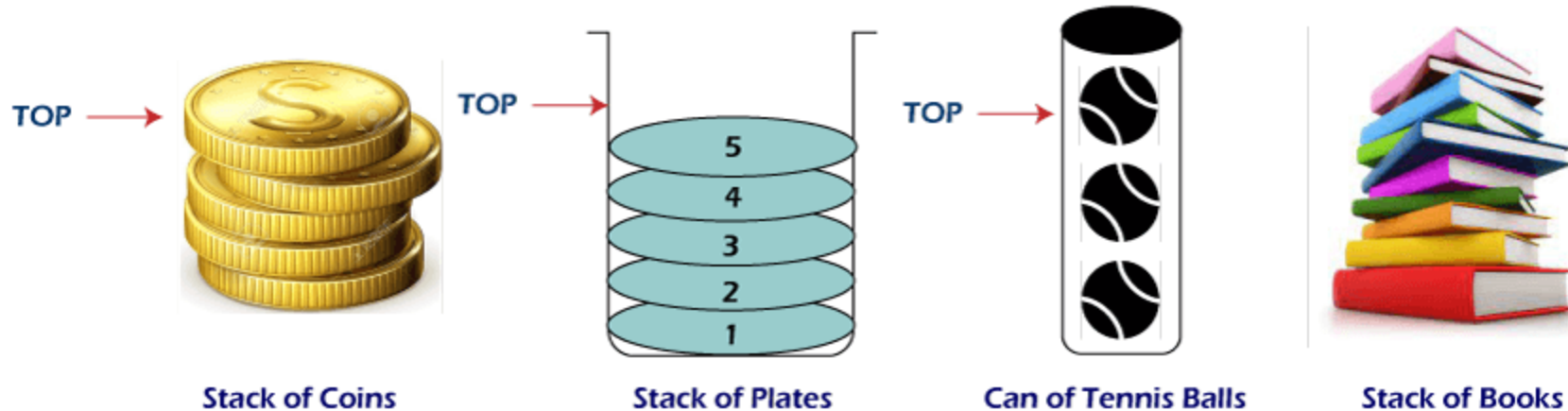
Types of Data Structure

- Broadly data structure categorized into two categories such as linear data structure and non-linear



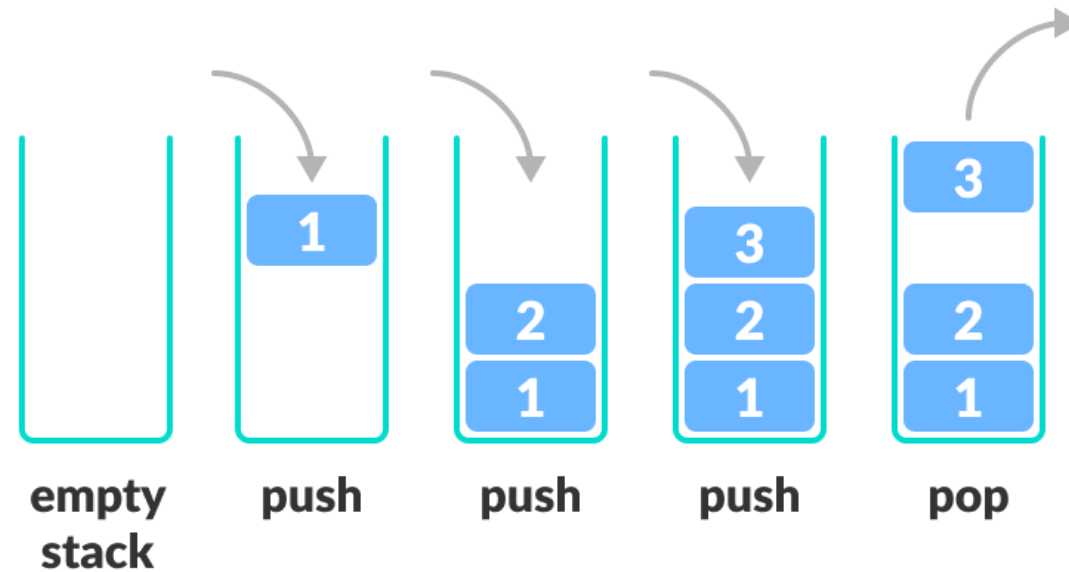
Stack

- A stack is a linear data structure that follows the principle of **Last In First Out (LIFO)**.
 - the last element inserted inside the stack is removed first.



Basic Operations of Stack

- **Push:** Add an element to the top of a stack
- **Pop:** Remove an element from the top of a stack
- **Peek:** Get the value of the top element without removing it
- **IsEmpty:** Check if the stack is empty



Queue

- Queue follows the **First In First Out (FIFO)** rule
 - the item that **goes in first** is the item that **comes out first**.



Basic Operations of Queue

- **Enqueue:** Add an element to the end of the queue
- **Dequeue:** Remove an element from the front of the queue
- **IsEmpty:** Check if the queue is empty
- **Peek:** Get the value of the front of the queue without removing it



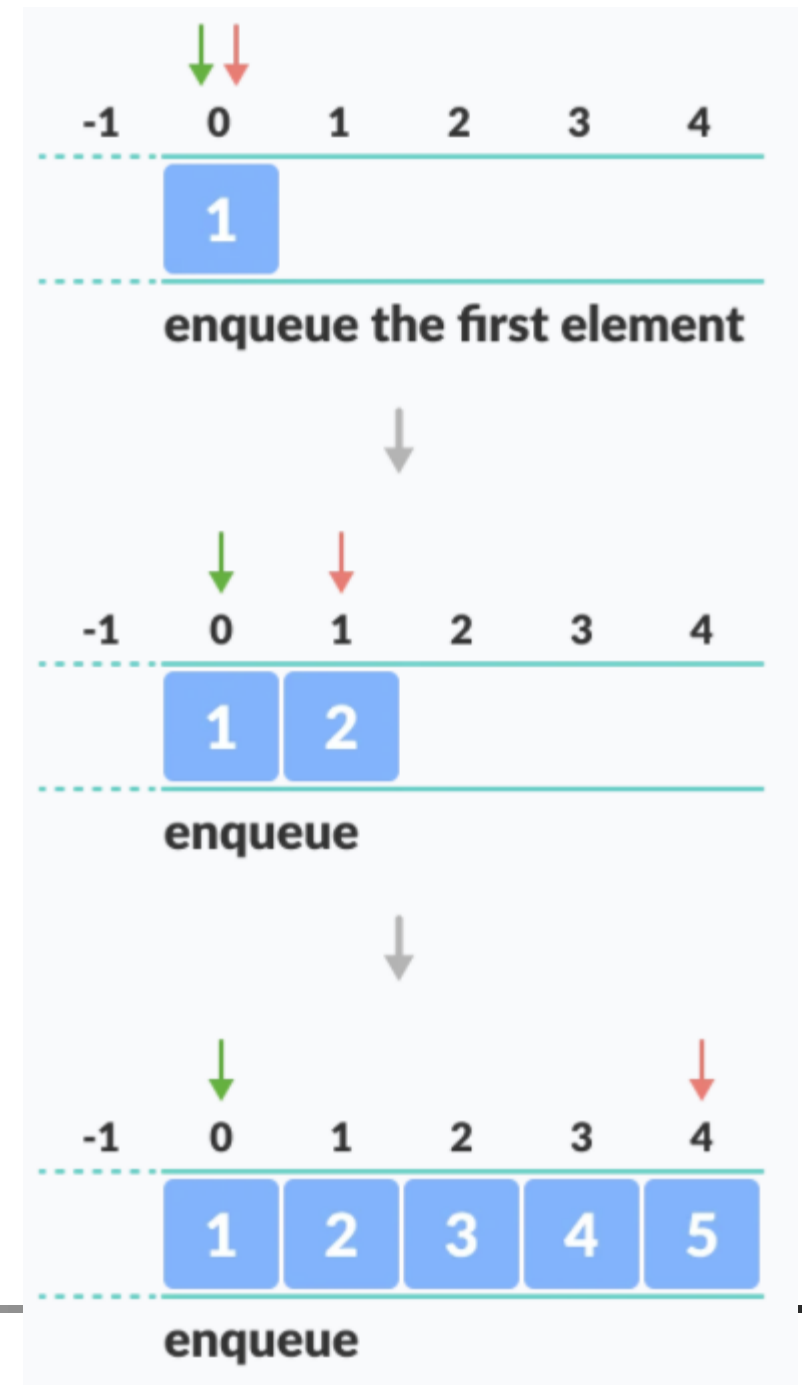
Working of Queue

- Queue operations work as follows:
 - two pointers **FRONT** and **REAR**
 - **FRONT** track the first element of the queue
 - **REAR** track the last element of the queue
 - initially, set value of **FRONT** and **REAR** to -1



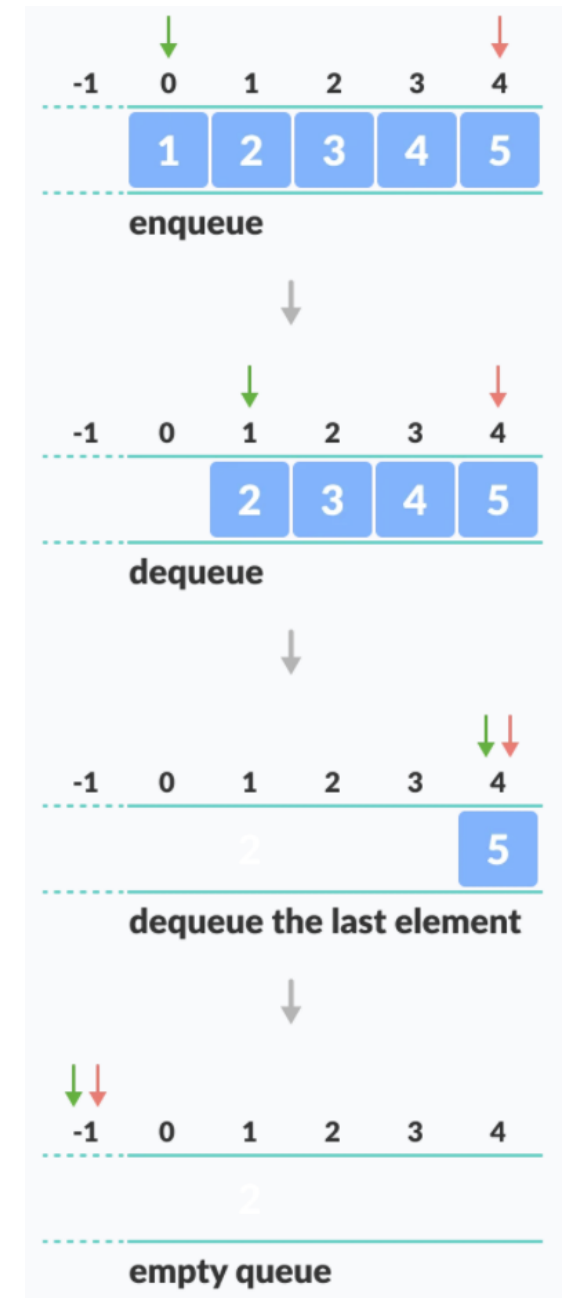
Enqueue

- For the first element, set the value of **FRONT** to 0
- Increase the **REAR** index by 1
- Add the new element in the position pointed to by **REAR**



Deque

- Check if the queue **is empty**
- Return the value pointed by **FRONT**
- **Increase** the **FRONT** index by 1
- For the last element, **reset** the values of **FRONT** and **REAR** to -1

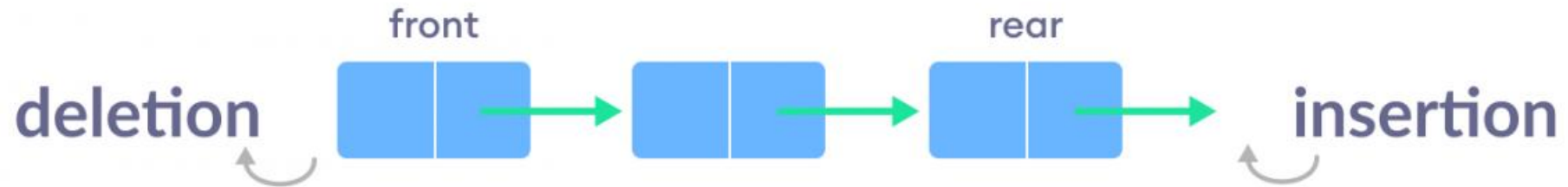


Types of Queues

- There are four different types of queues:
 - Simple Queue
 - Circular Queue
 - Priority Queue
 - Double Ended Queue

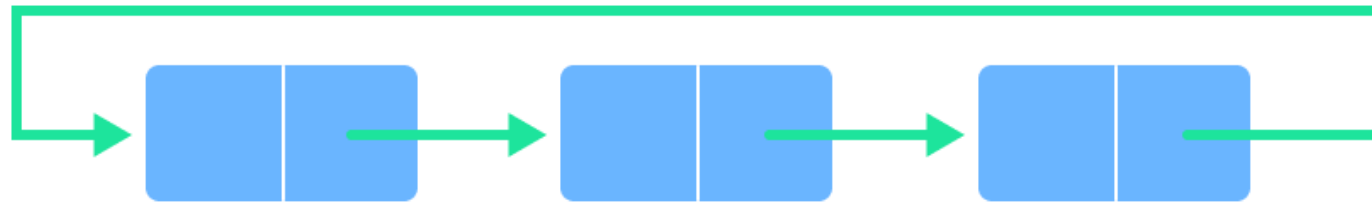
Simple Queue

- In a simple queue, insertion takes place at the rear and removal occurs at the front. It strictly follows the FIFO (First in First out) rule.



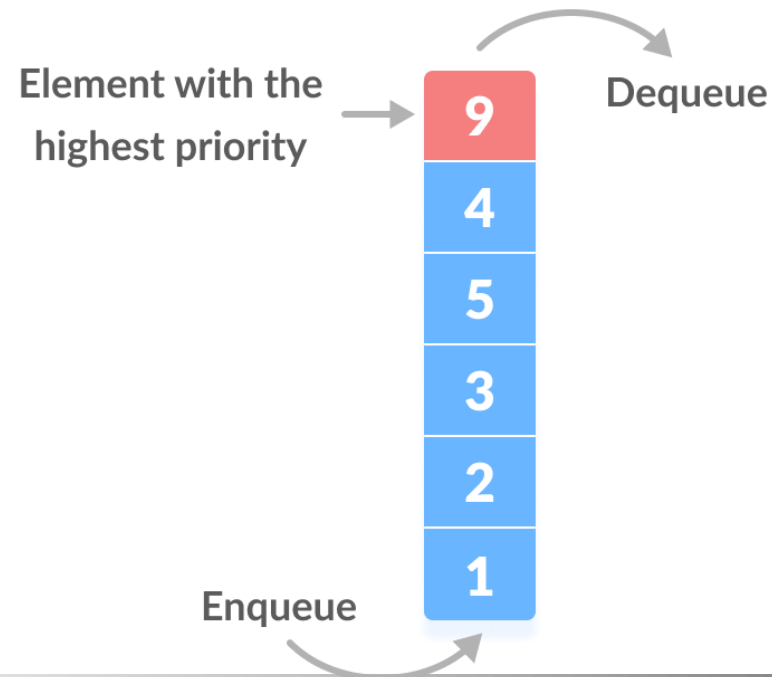
Circular Queue (circular buffer)

- In a circular queue, the **last element points to the first element** making a circular link.
 - The main advantage of a circular queue over a simple queue is **better memory utilization**. If the last position is full and the first position is empty, we can insert an element in the first position.



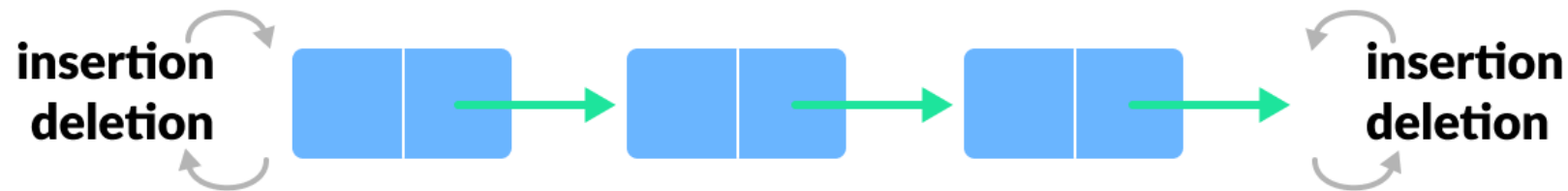
Priority Queue

- A priority queue is a special type of queue in which each element is associated with a **priority value**. And, elements are served on the basis of their priority. That is, higher priority elements are served first.



Double Ended Queue

- In a double ended queue, insertion and removal of elements can be performed from either from the front or rear. Thus, it does not follow the FIFO (First In First Out) rule.

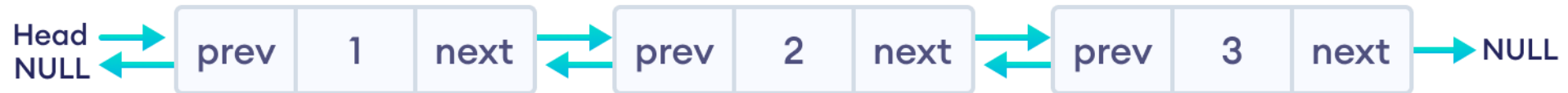


Linked List

- A linked list is a linear data structure that includes a series of connected **nodes**.
- Unlike an array, the order in a linked list is determined by a **pointer** in each object.
- Linked lists can be of multiple types: singly, doubly, and circular linked list.
 - In this Lecture, we will focus on the **doubly linked list**.

Doubly Linked List

- A doubly linked list is a type of linked list in which each node consists of 3 components:
 - **prev** - address of the previous node
 - **data** - data item
 - **next** - address of next node



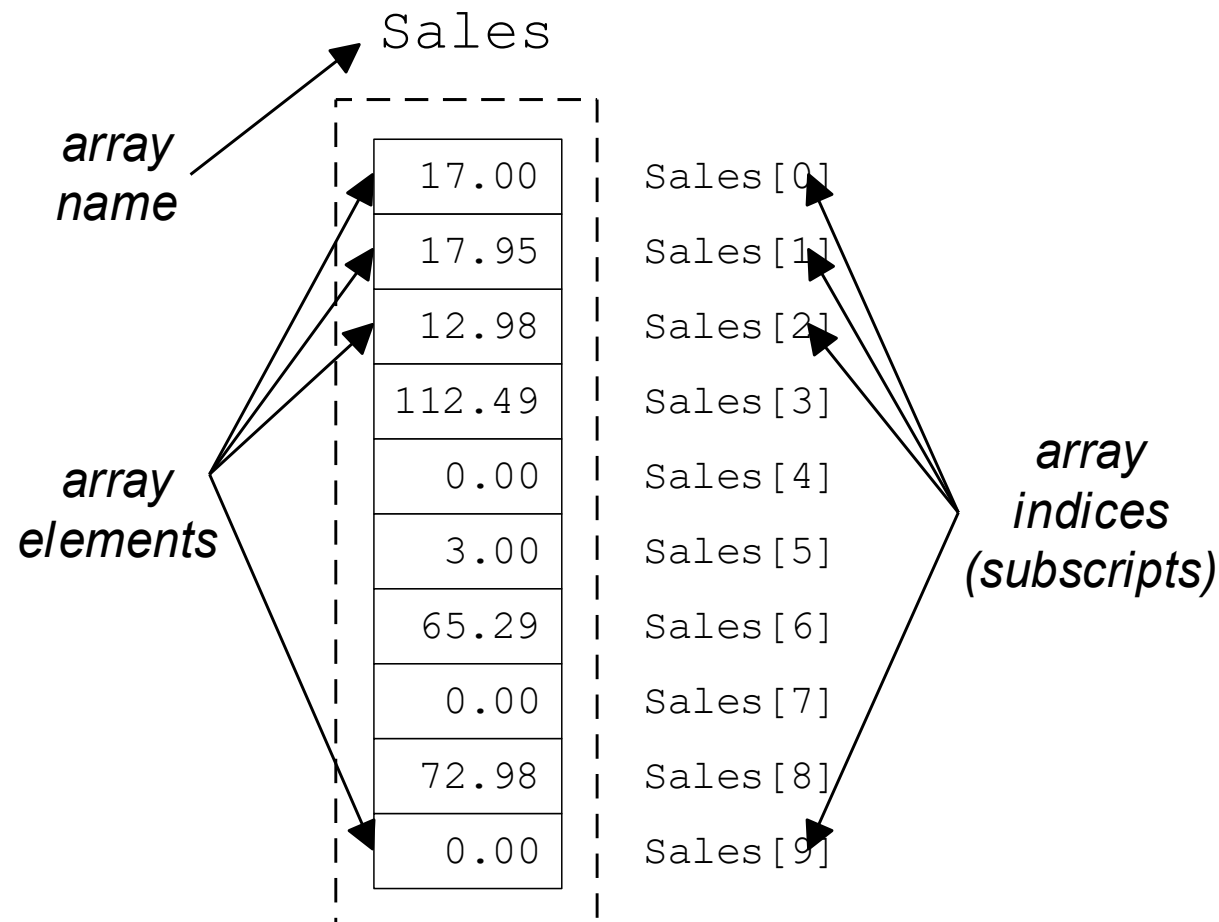
- **head** points to the first node of the linked list.
- **next** pointer of the **last node** is NULL.

Operations of Linked List

- **Traversal** - access each element of the linked list
- **Insertion** - adds a new element to the linked list
- **Deletion** - removes the existing elements
- **Search** - find a node in the linked list

Array

- Arrays store elements in **contiguous memory locations**, resulting in easily calculable addresses for the elements stored and this allows faster access to an element at a specific index.



Array vs. Linked List

Array	Linked List
Arrays are stored in contiguous location.	Linked lists are not stored in contiguous location.
Fixed in size	Dynamic in size
Memory is allocated at compile time	Memory is allocated at run time
Uses less memory than linked lists	Uses more memory because it stores both data and address of next node
Elements can be accessed easily	Element accessing requires the traversal of whole linked list
Insertion and deletion operation takes time	Insertion and deletion operation is faster

Singly Linked List



Singly Linked List vs. doubly Linked List

	Singly Linked List (SLL)	Doubly Linked List (DLL)
Fields	data and next.	data, prev and next.
Traversal direction	one direction	both directions
Memory	Less memory	More memory (3 fields)
Complexity of: insertion and deletion at a given position	$O(n)$	$O(n)$
Find next element	$O(1)$	$O(1)$
Find previous element	$O(n)$	$O(1)$
deletion with a given node	$O(n)$	$O(1)$
Insert a new node before a given node	$O(n)$	$O(1)$

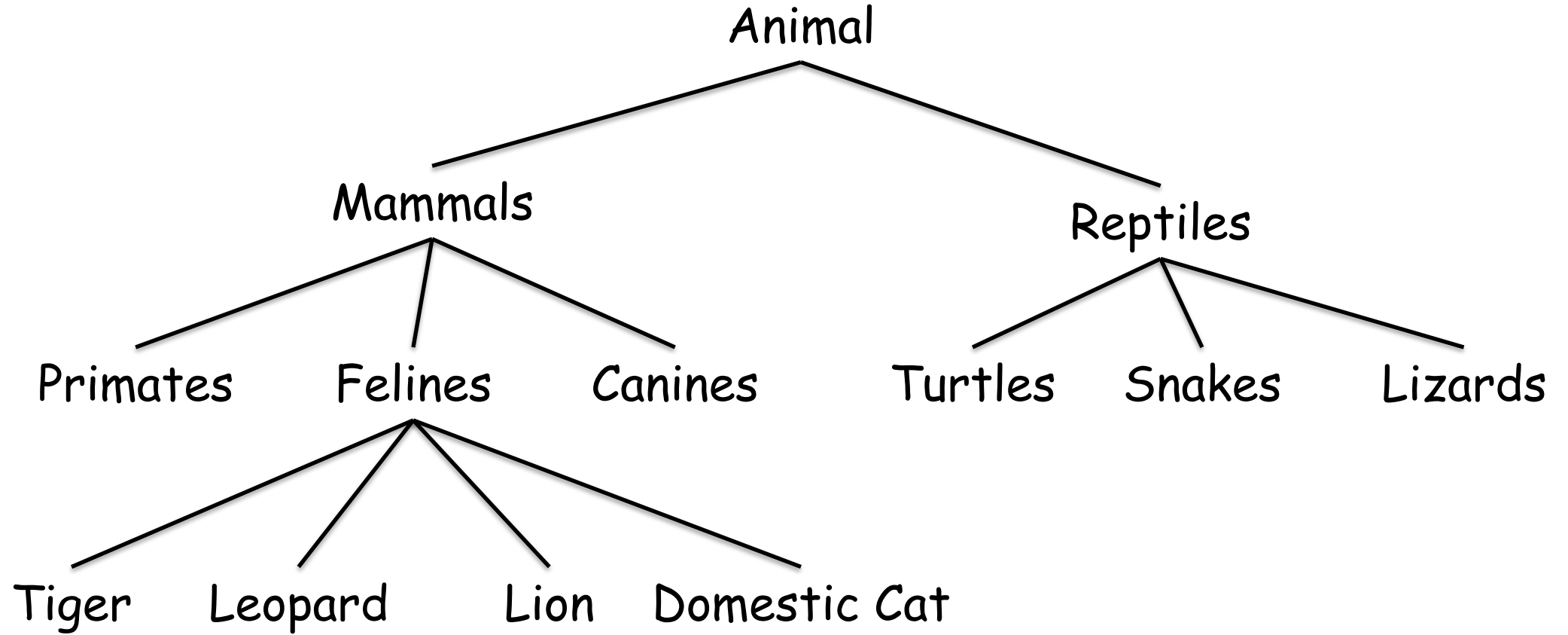
Tree Data Structure

- A tree is a **nonlinear hierarchical data structure** that consists of **nodes** connected by **edges**.
- Why tree data Structure?
 - Other data structures such as arrays, linked list, stack, and queue are **linear** data structures that store data sequentially.
 - Different tree data structures allow **quicker and easier access** to the data as it is a **non-linear** data structure.

Tree

- Some data are not linear (it has more structure!)
 - Family trees
 - Organizational charts
 - ...
- Trees offer an alternative
 - Representation
 - Implementation strategy
 - Set of algorithms

Example: Taxonomy Tree



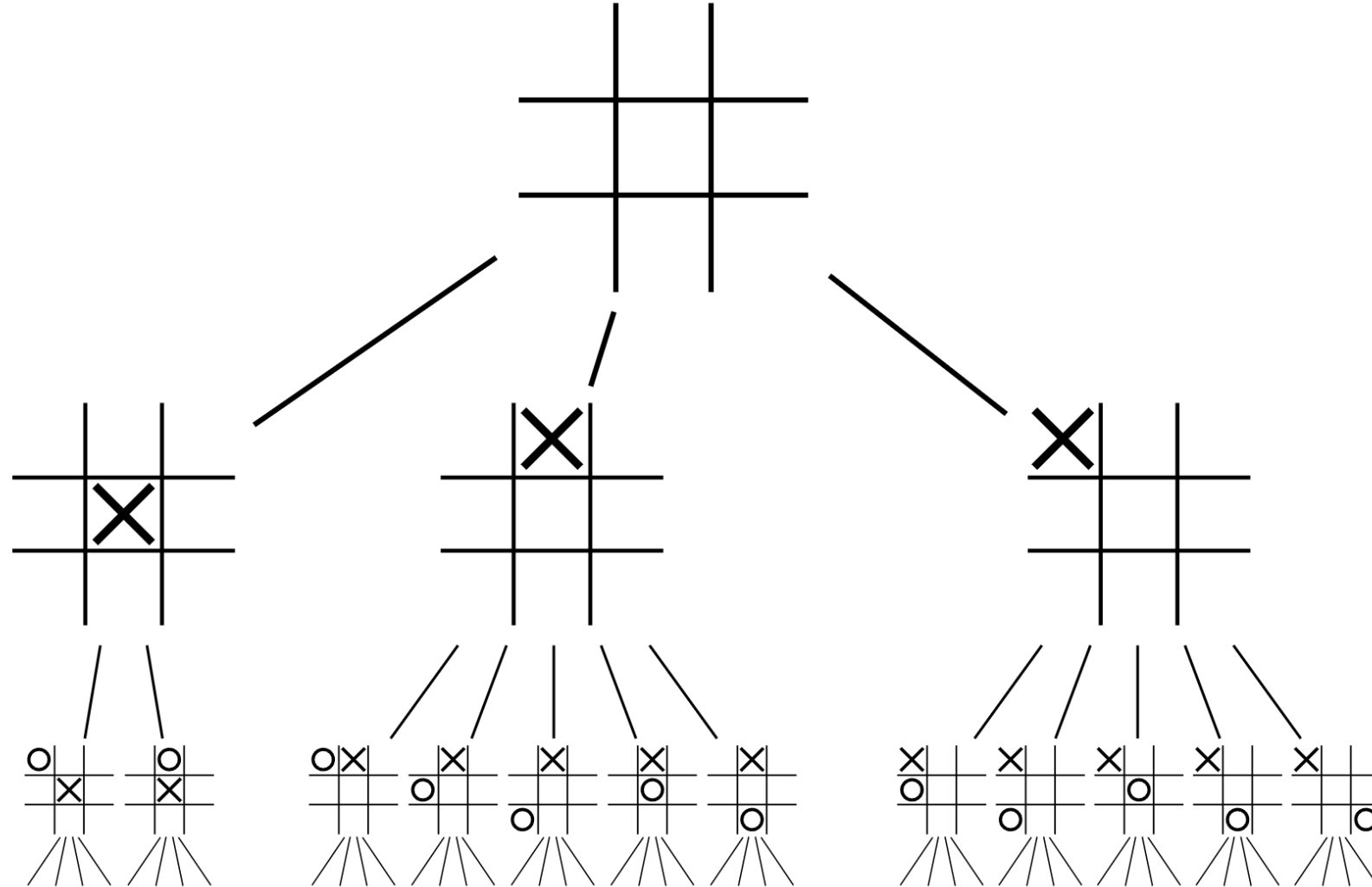
How many types of tree are there?

- Far too many:
 - General Tree
 - Binary Tree
 - Red-Black Tree
 - AVL Tree
 - B+ Tree
 - ...
- Different types are used for different things
 - To improve speed
 - To improve the use of available memory
 - To suit particular problems

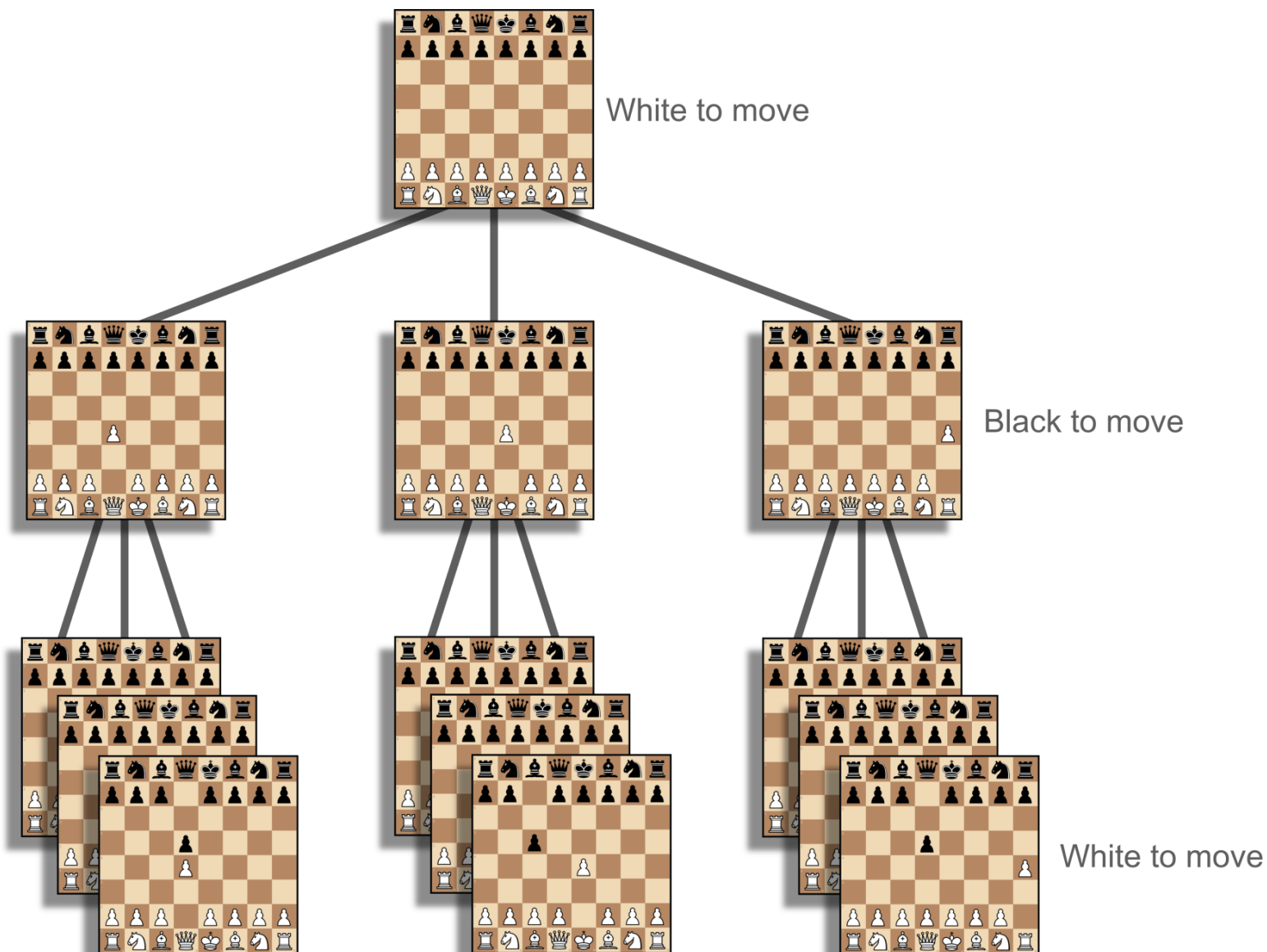
What is a tree useful for?

- Artificial Intelligence - planning, navigating, games
- Representing things:
 - Simple file systems
 - Class inheritance and composition
 - Classification, e.g., taxonomy
 - HTML pages
 - Parse trees for languages
 - Essential in compilers like java
 - Etc.

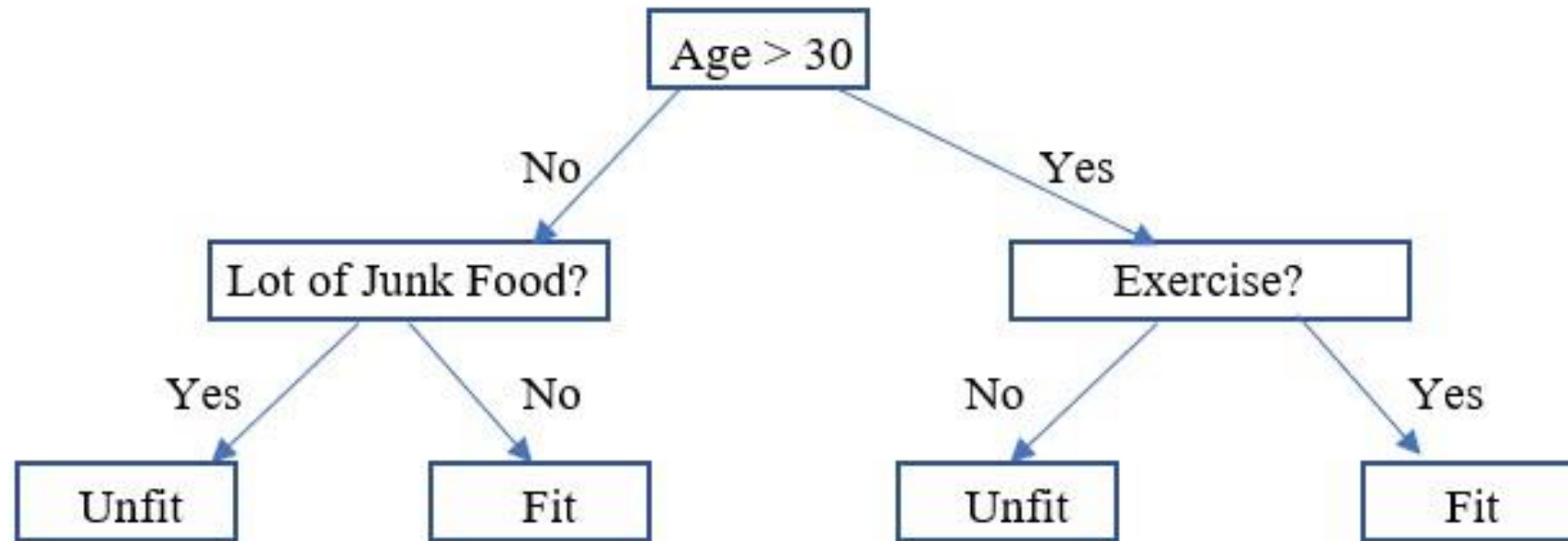
Example: Tic Tac Toe



Example: Chess



Example: Decision Tree

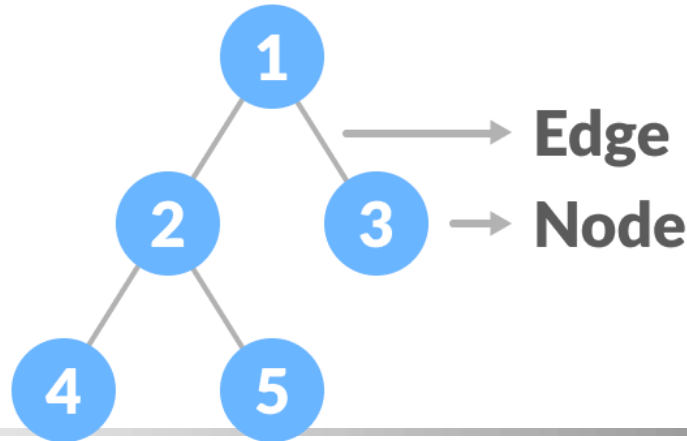


Tree Terminologies

■ Node

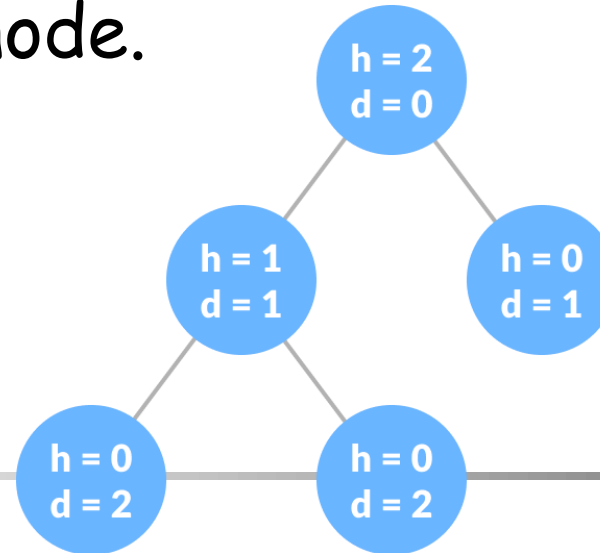
- A node is an entity that contains a **key** and **pointers to its child nodes**.
- The last nodes of each path are called **leaf nodes** or external nodes that do not contain a pointer to child nodes.
- The node having at least a child node is called an **internal node**.

■ Edge is the link between any two nodes.



Tree Terminologies

- **Root** is the topmost node of a tree
- The **Height of a node** is the number of edges from the node to the deepest leaf
- The **Depth (or Level) of a Node** is the number of edges from the root to the node.
- The **height of a Tree** is the height of the root node or the depth of the deepest node.



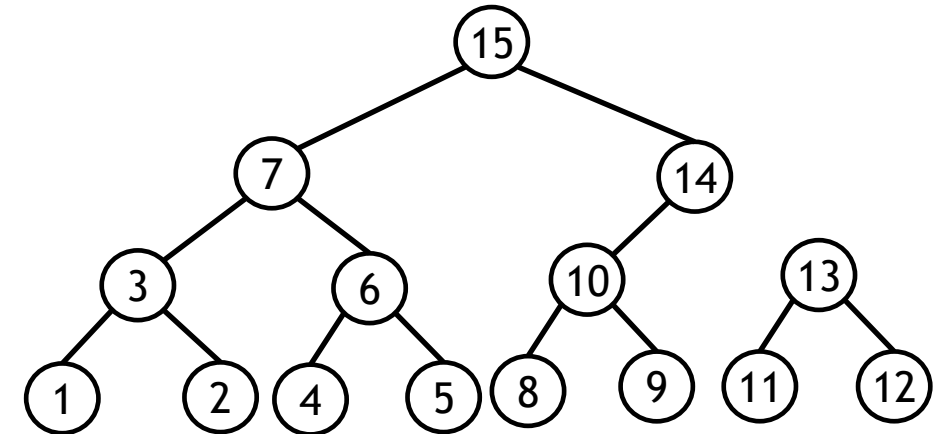
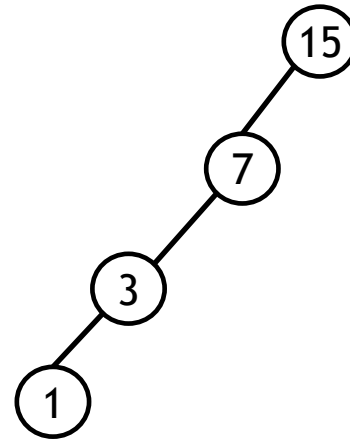
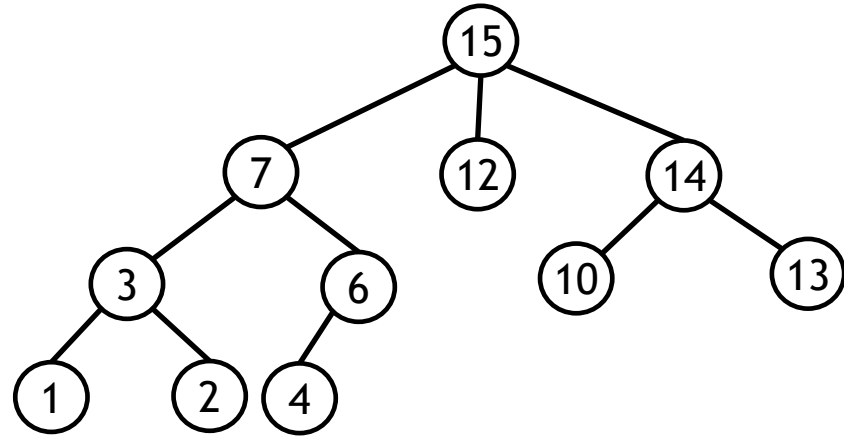
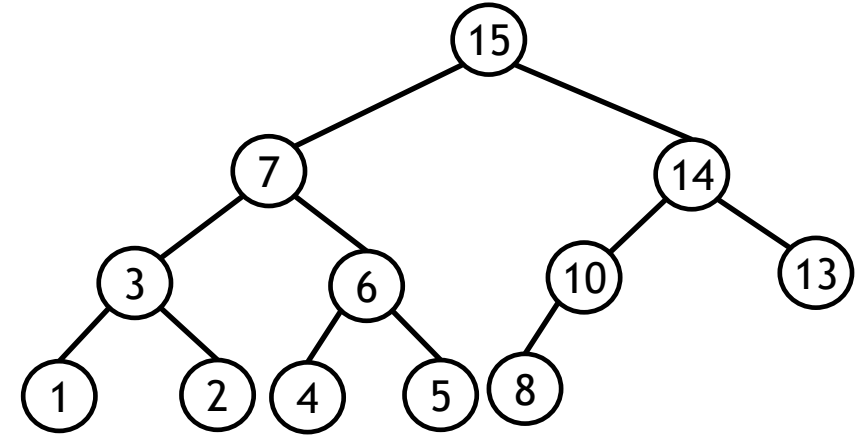
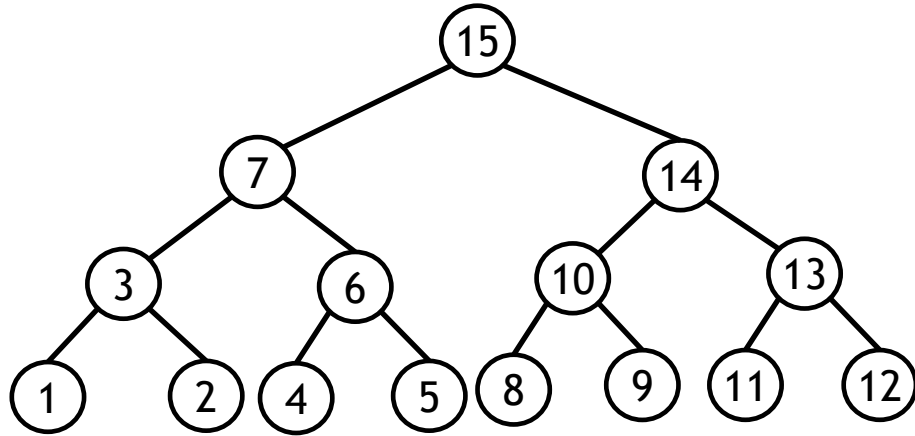
Tree Terminologies

- The **degree of a node** is the total **number of branches** of that node.
 - Binary tree: each node has at most **two children**.
- A collection of disjoint trees is called a **forest**.

Binary Tree

- Binary tree is a **tree data structure**, each node has at most two **children** (*left child and right child*).
- **Complete (perfect) binary tree**
 - all interior nodes have **two children**
 - all leaves have the **same depth or same level**.
- **Nearly complete binary tree**
 - every level, except possibly the last, is completely filled.
 - all nodes in the last level are as far left as possible.

Is this Binary Tree?

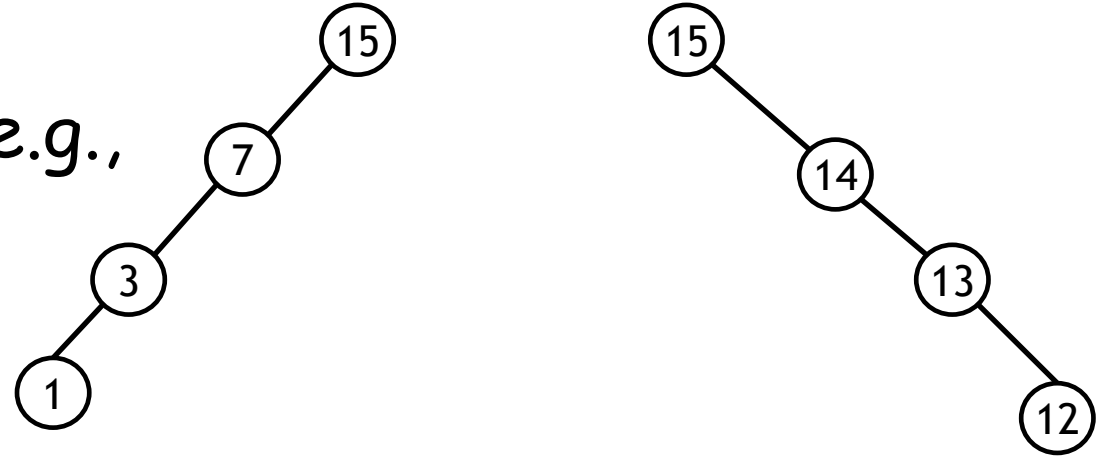


Worst Case

- Operations can degenerate to $O(n)$ - worst case!

- Degenerates to a linked list, e.g.,

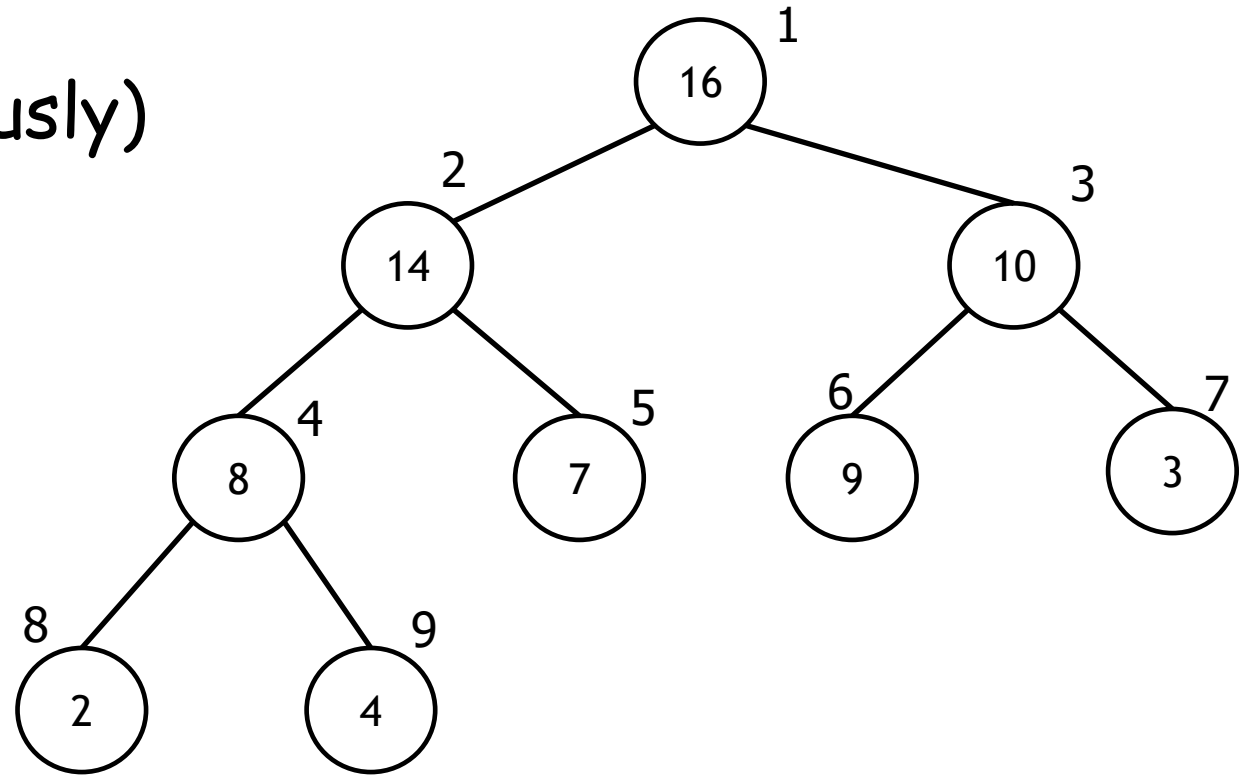
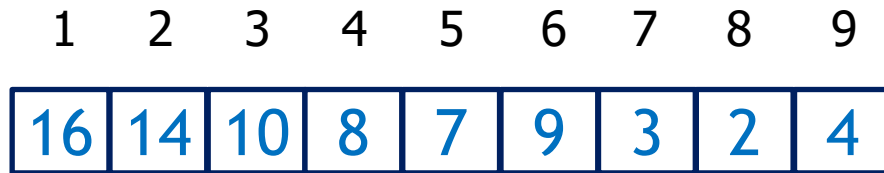
- All nodes are to the left
- All nodes are to the right



- Balance the tree to guarantee that height is $O(\log n)$.

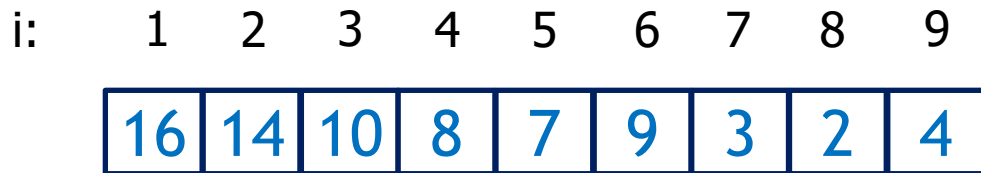
Heap

- An **array**, visualized as a nearly complete binary tree.
- **Max Heap Property**: The key of a node is $\geq$ than the keys of its children
- (**Min Heap** defined analogously)

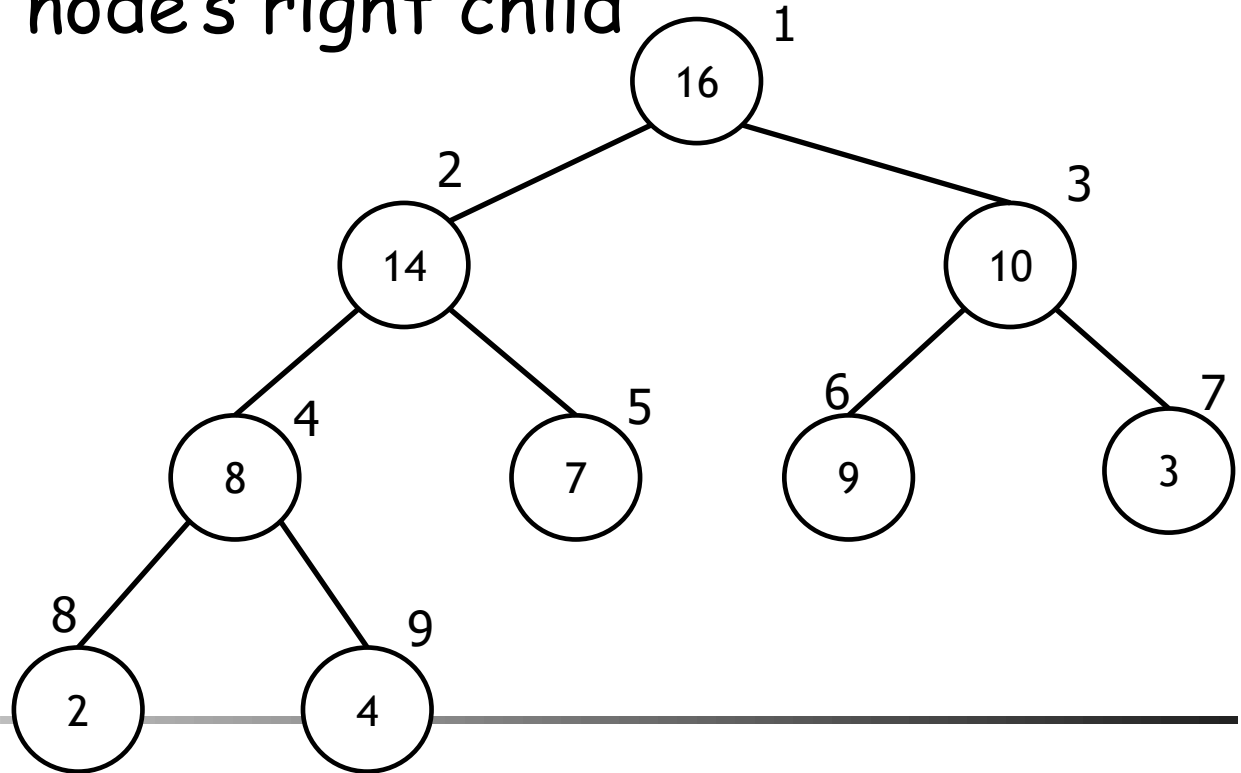


Heap as a tree

- **Root of tree**: first element in the array, corresponding to $i = 1$
- **Parent(i)**= $i/2$: returns index of node's parent
- **left(i)**= $2i$: returns index of node's left child
- **right(i)**= $2i+1$: returns index of node's right child



No pointers required!
Height of a binary heap is $O(\log n)$



Heap Operations

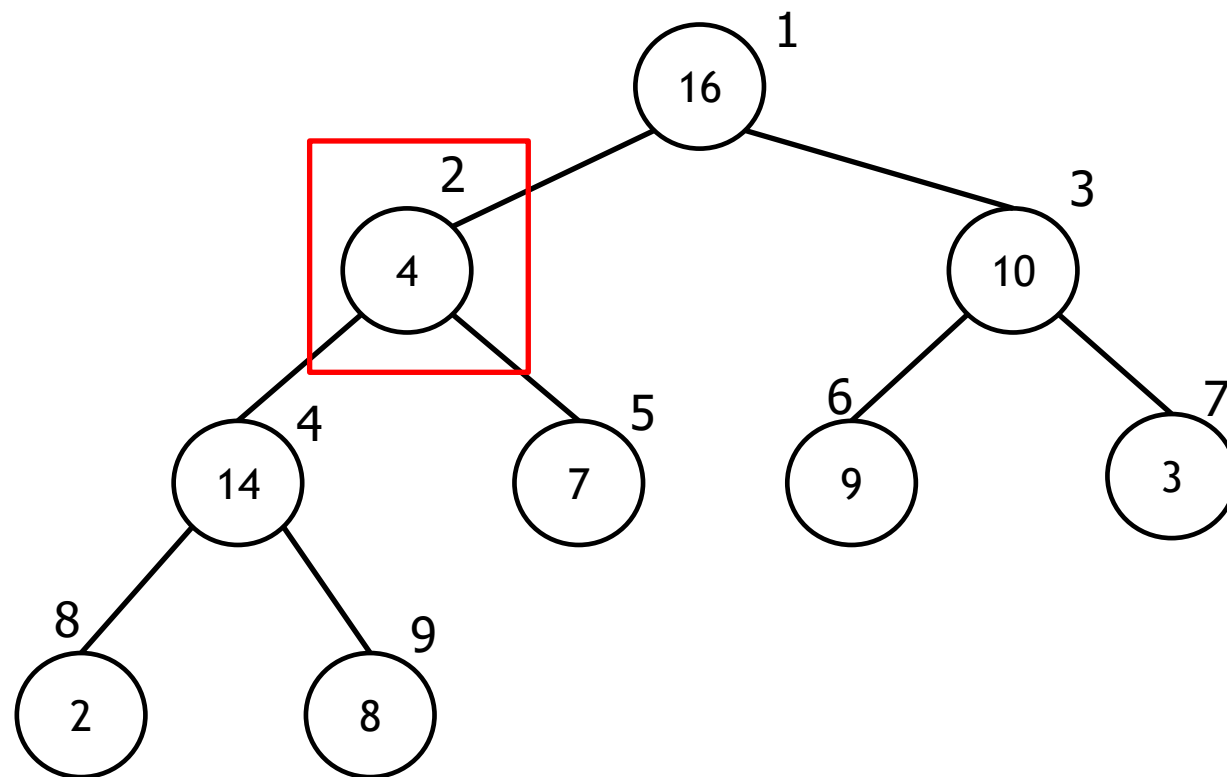
- **Max_heapify**: correct a single violation of the heap property in a subtree at its root
- **Build_max_heap**: produce a max-heap from an unordered array
- **Insertion**: add a new item in the heap
- **Deletion**: delete an item from the heap

Heapify

- If element $A[i]$ violates the max-heap property, **MAX-HEAPIFY** lets the value at $A[i]$ “float down” in the max-heap so that the subtree rooted at index i obeys the max-heap property.

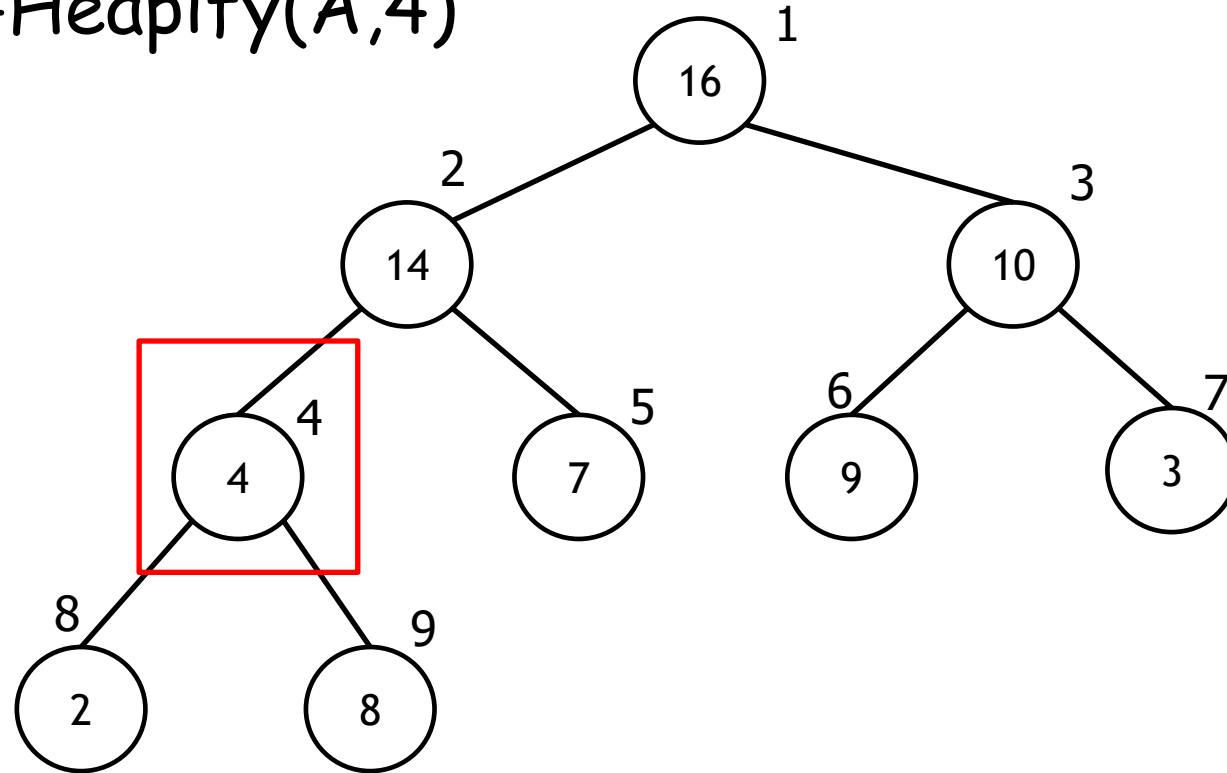
Heapify

- $\text{MAX-Heapify}(A, 2)$



Heapify

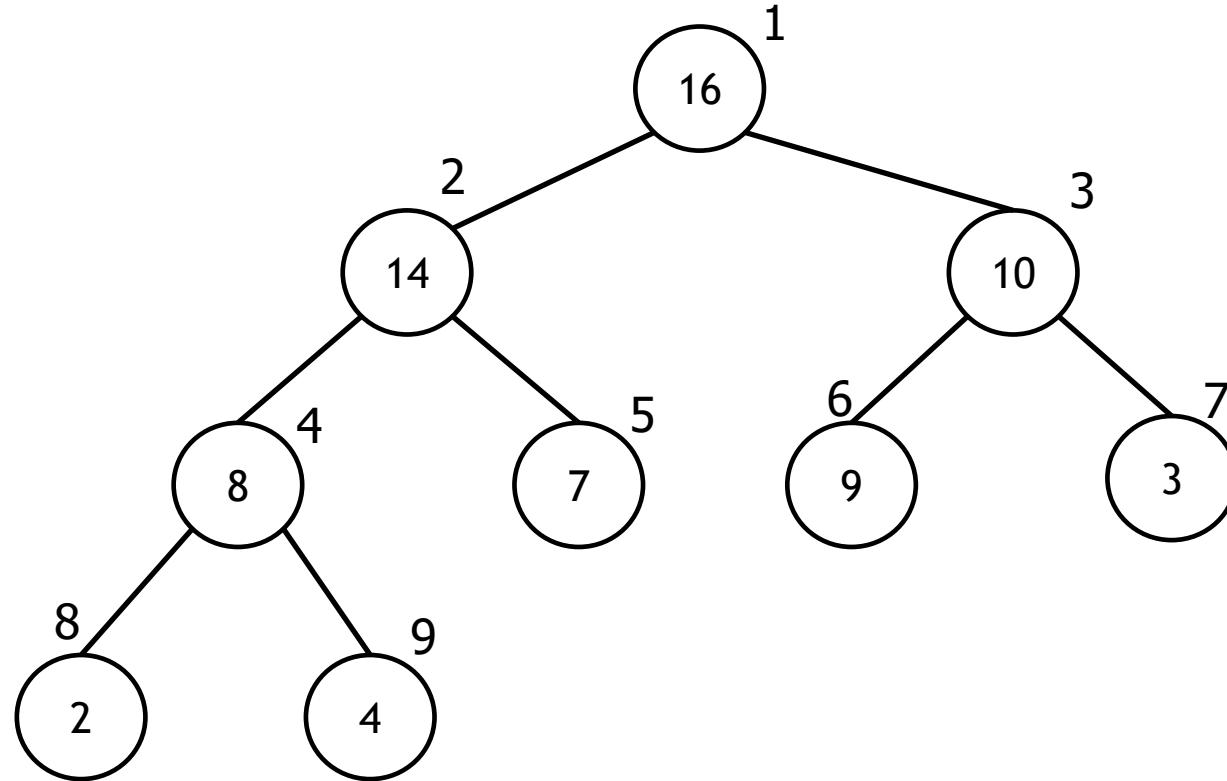
- Exchange $A[2]$ with $A[4]$
- Then, call $\text{MAX-Heapify}(A, 4)$



Heapify

- Time Complexity ?

$O(\log n)$



MAX-HEAPIFY Pseudocode

Max_Heapify(A,i):

l = left(i)

r = right(i)

if (l <= heap-size(A) and $A[l] > A[i]$)

largest = l

else largest = i

if (r <= heap-size(A) and $A[r] > A[largest]$)

largest = r

if largest $\neq$ i

exchange $A[i]$ and $A[largest]$

Max_Heapify(A, largest)

Build_Max_Heap(A)

- Converts $A[1...n]$ to a max heap

Build_Max_heap(A):

for $i=n/2$ down to 1

do Max_heapify(A,i)

- Why start at $n/2$?
 - $A[n/2+1,...n]$ are all leaves of the tree
- Time Complexity?
 - $O(n \log n)$???

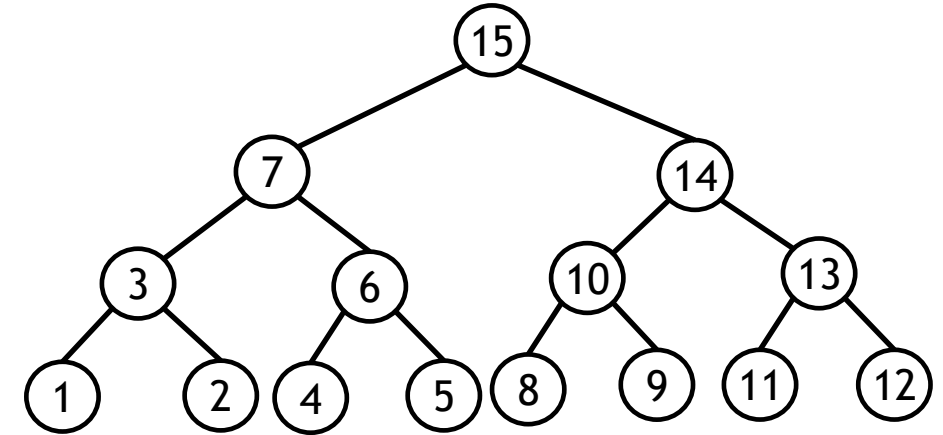
Build_Max_Heap(A) Analysis

- Converts $A[1..n]$ to a max heap

Build_Max_heap(A):

for $i = n/2$ down to 1

do Max_heapify(A,i)



- Max_heapify takes
 - $O(1)$ time for nodes that are one level above the leaves
 - $O(k)$ time for the nodes that are k levels above the leaves
- How many nodes we have in each level?
 - 1 node, 2 nodes, 4 nodes, ..., $n/4$, $n/2$ nodes

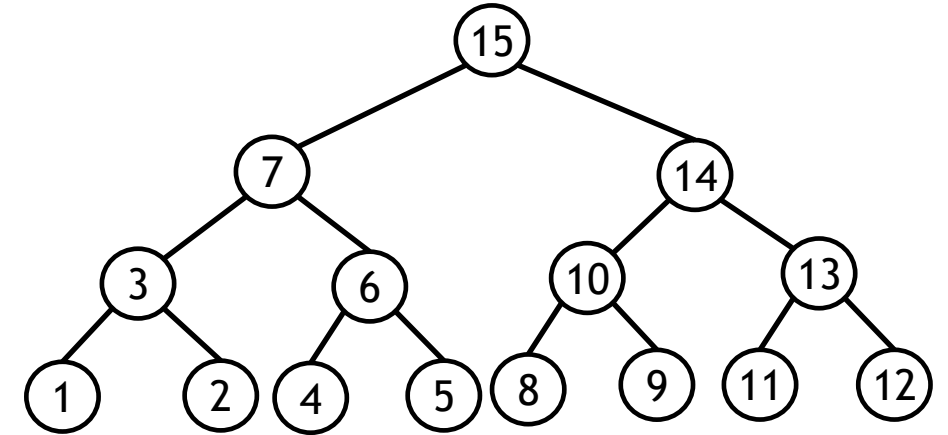
Build_Max_Heap(A) Analysis

- Converts $A[1..n]$ to a max heap

Build_Max_heap(A):

for $i=n/2$ down to 1

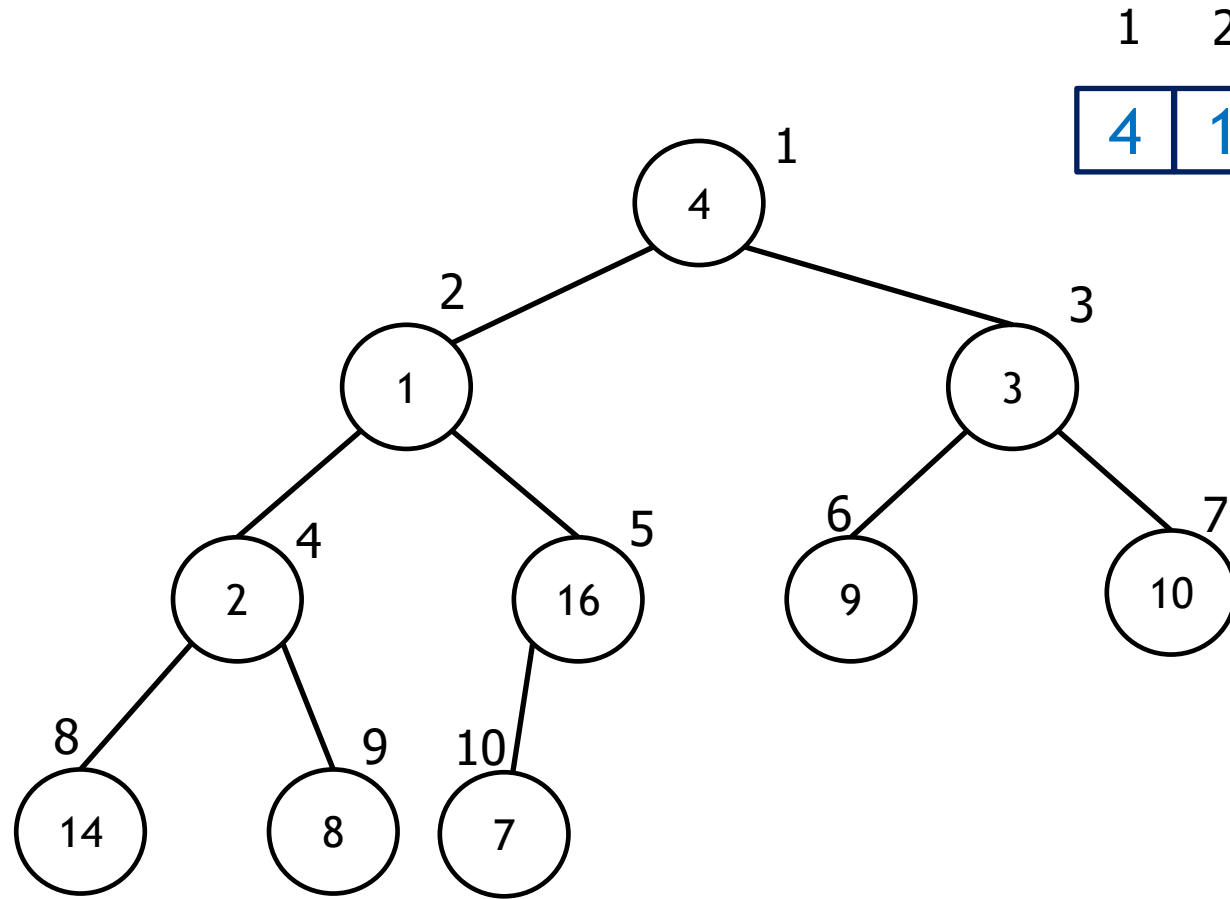
do Max_heapify(A,i)



- Total amount of work in the for loop:
- $T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil O(h) = O\left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h}\right)$
- Build Max Heap is $O(n)$.

Bounded by a Constant!

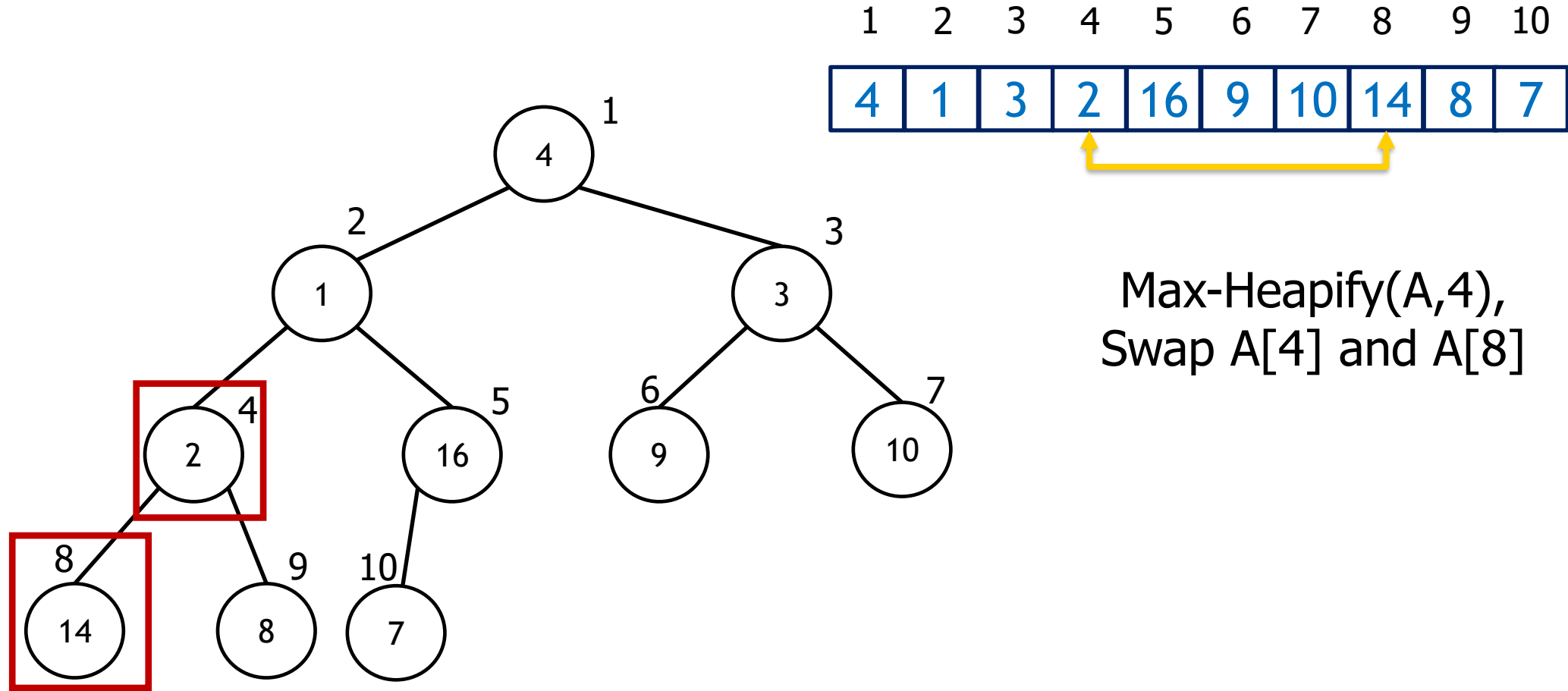
Build_Max_Heap



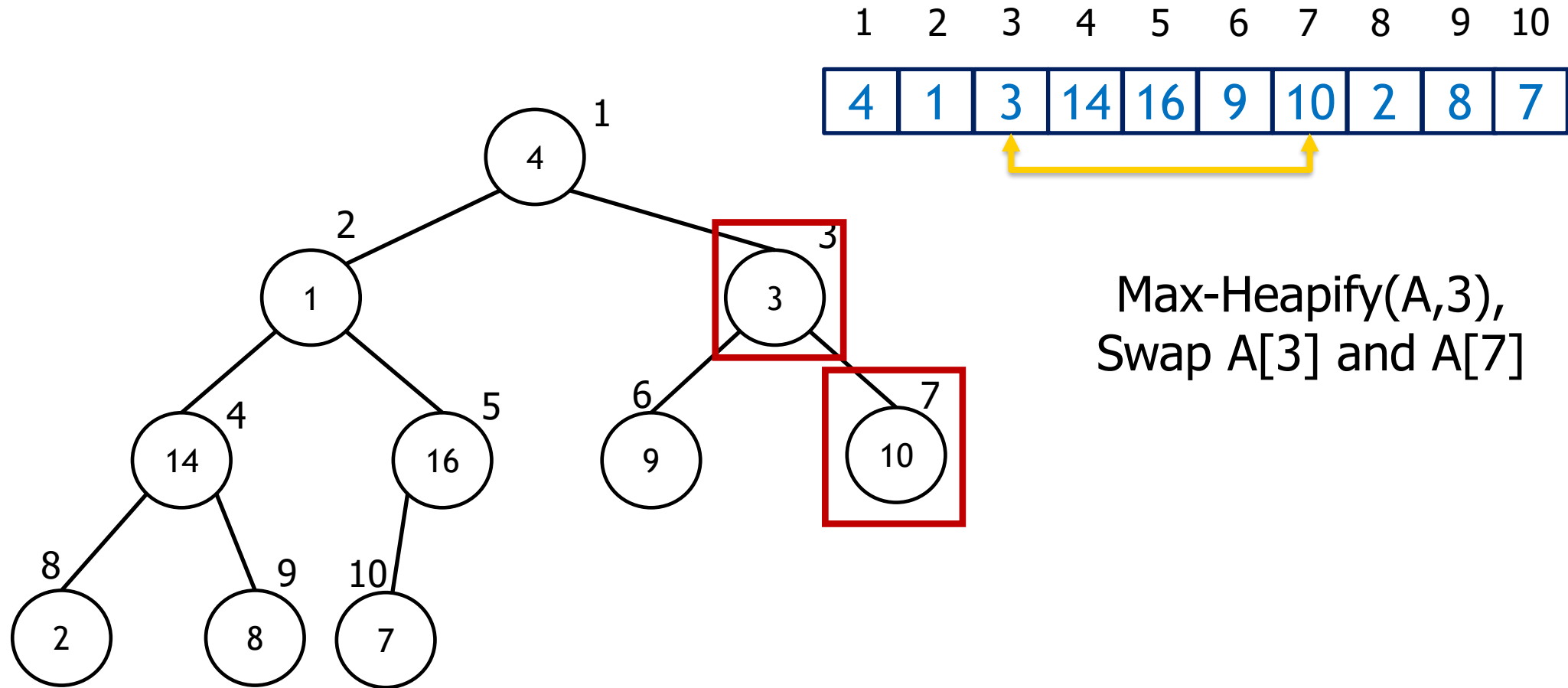
1	2	3	4	5	6	7	8	9	10
4	1	3	2	16	9	10	14	8	7

Max-Heapify(A,5), no change

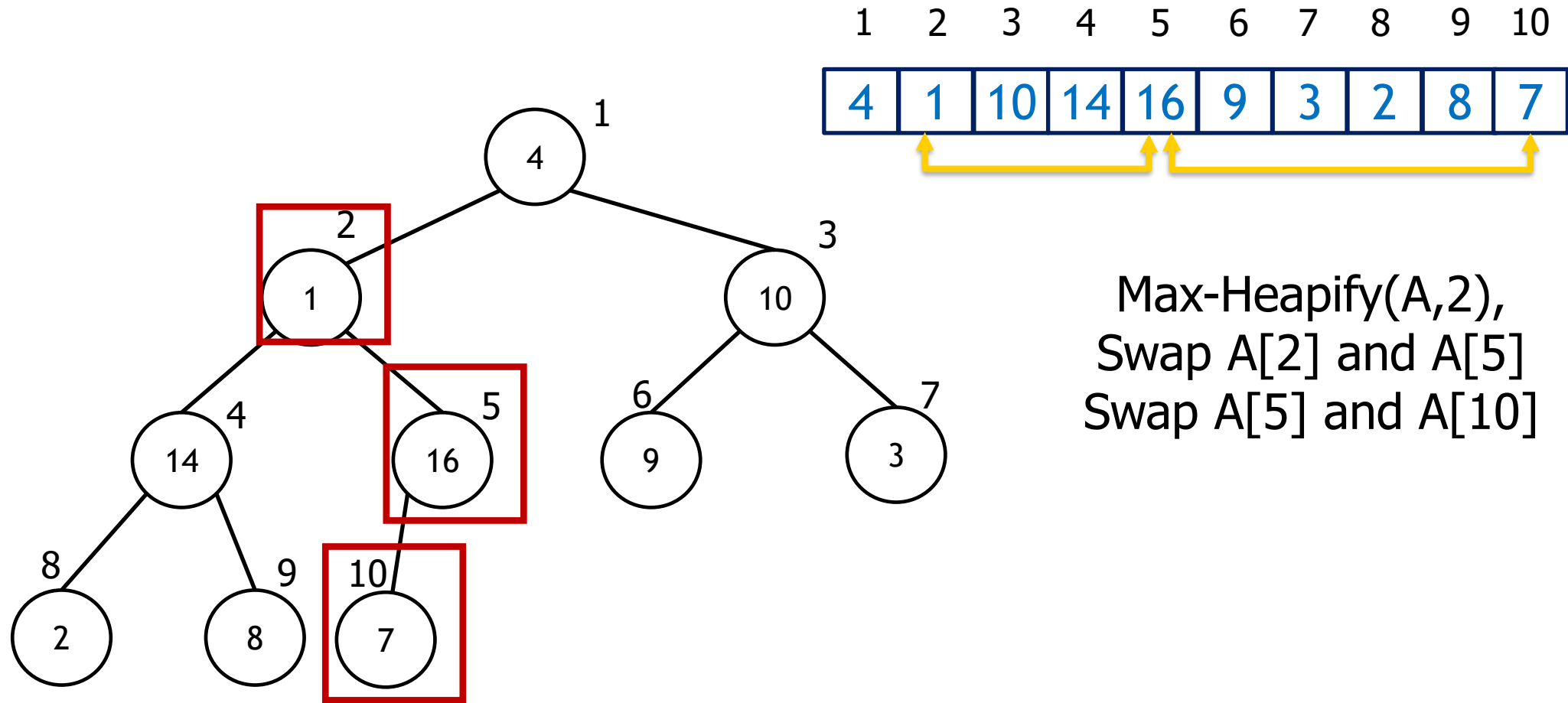
Build_Max_Heap



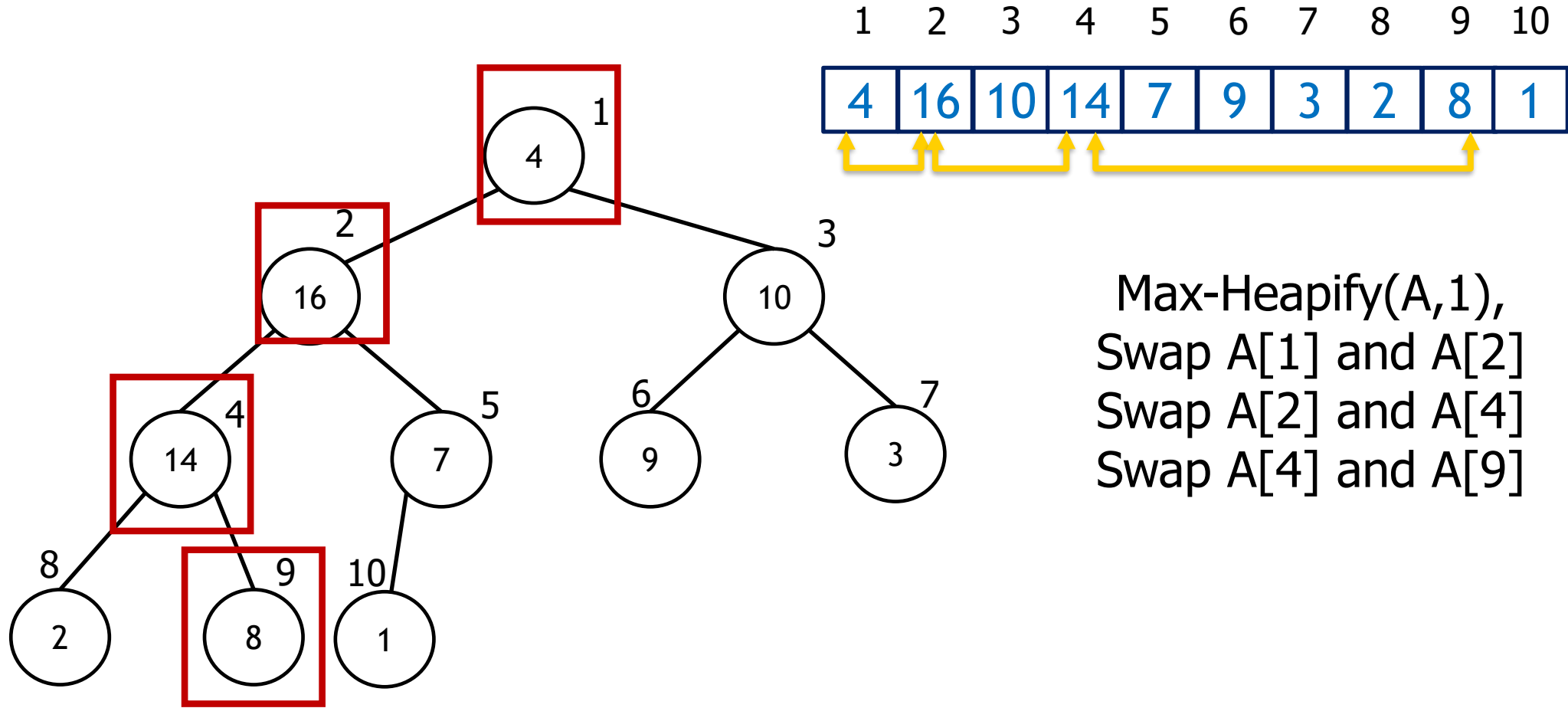
Build_Max_Heap



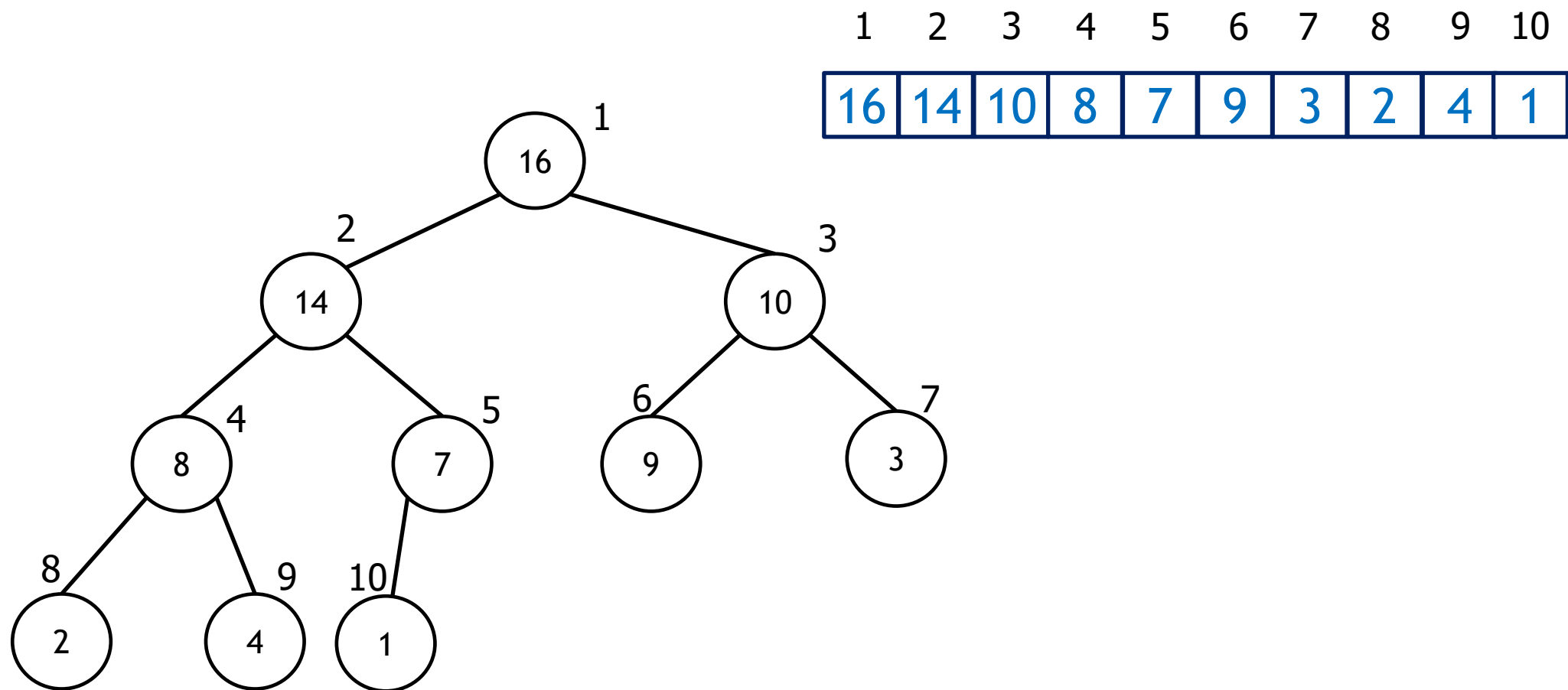
Build_Max_Heap



Build_Max_Heap



Build_Max_Heap

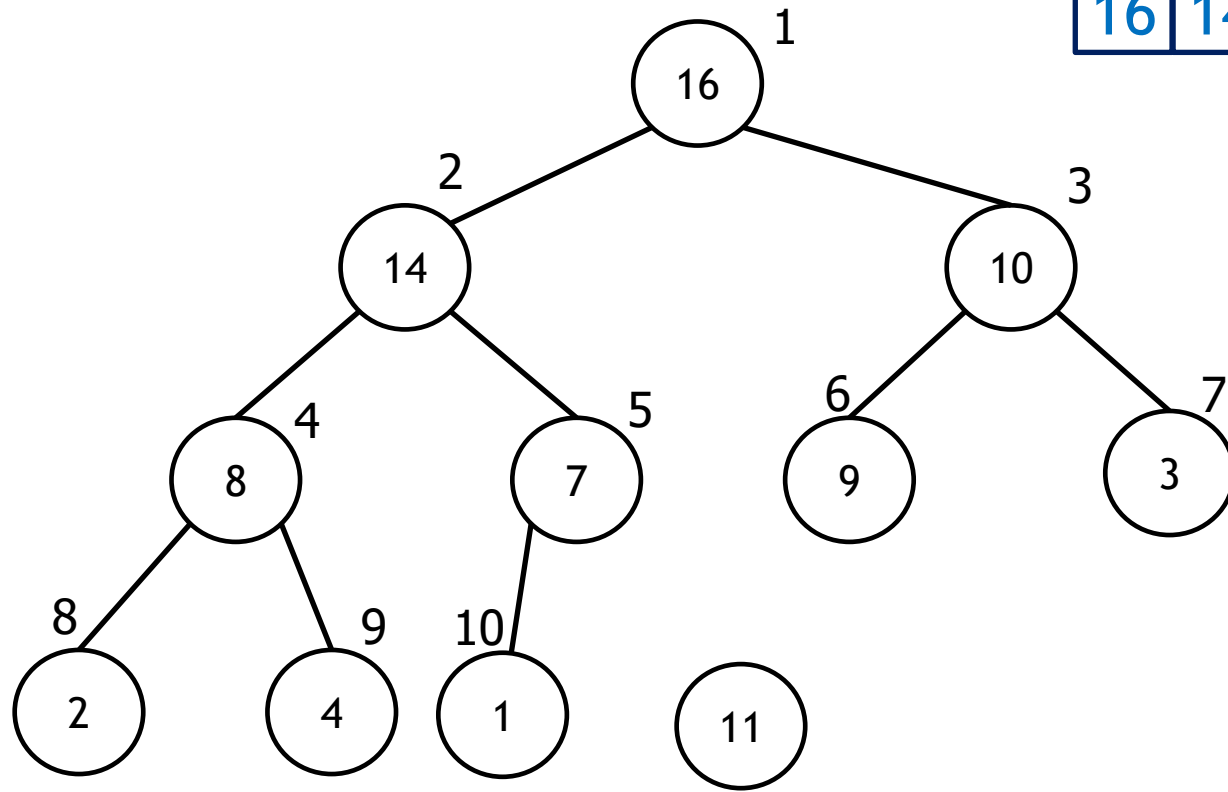


Heap Insertion

- To add an element to a heap:
 - 1) Add the element to the **bottom level** of the heap at the **leftmost open space**.
 - 2) Compare the **added element** with its **parent**; if they are in the **correct order, stop**.
 - 3) **If not, swap** the element with its **parent** and **return to the previous step**.

Heap Insertion

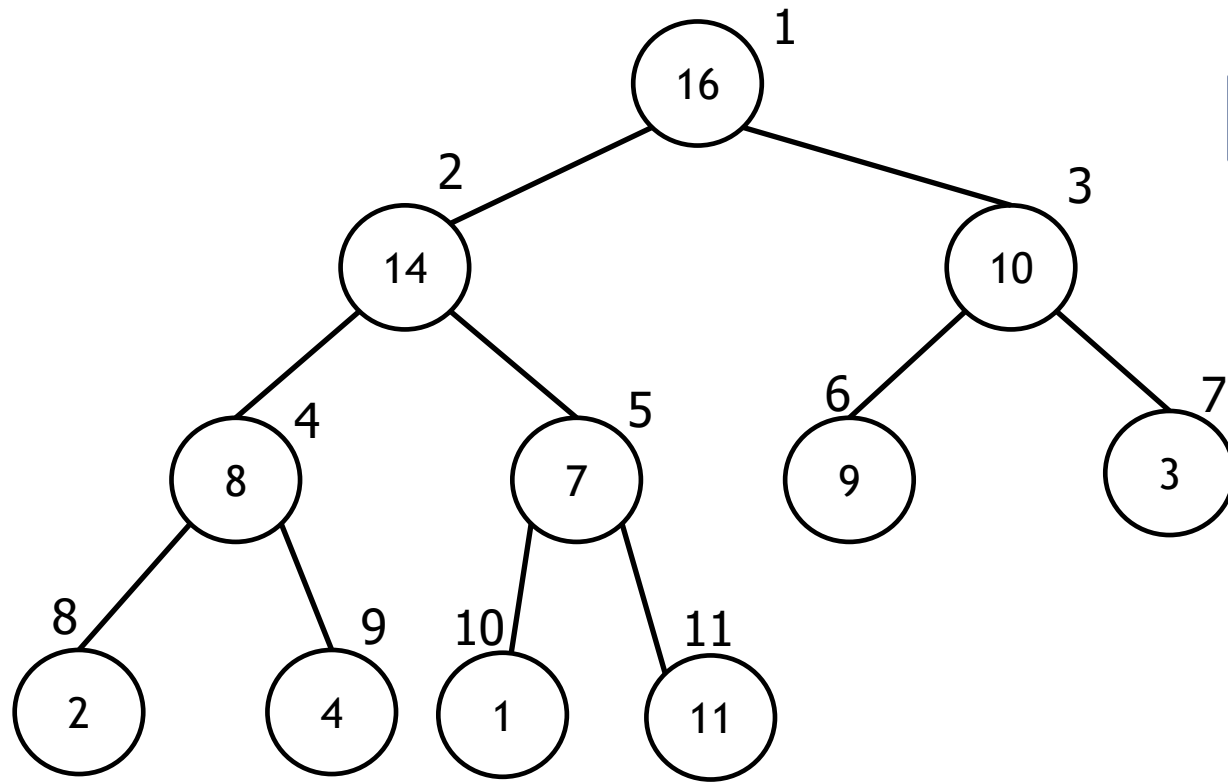
■ Insert 11



1	2	3	4	5	6	7	8	9	10	
16	14	10	8	7	9	3	2	4	1	11

Heap Insertion

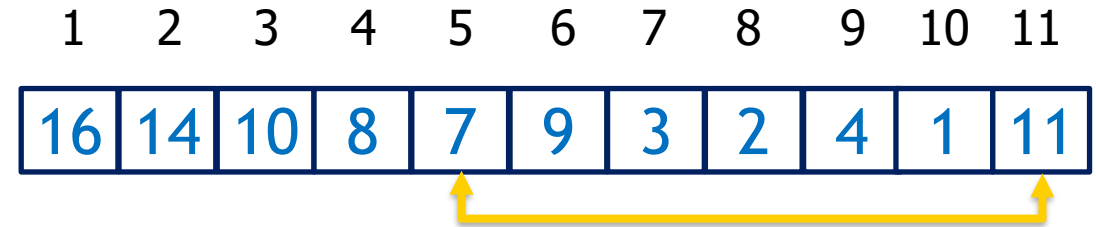
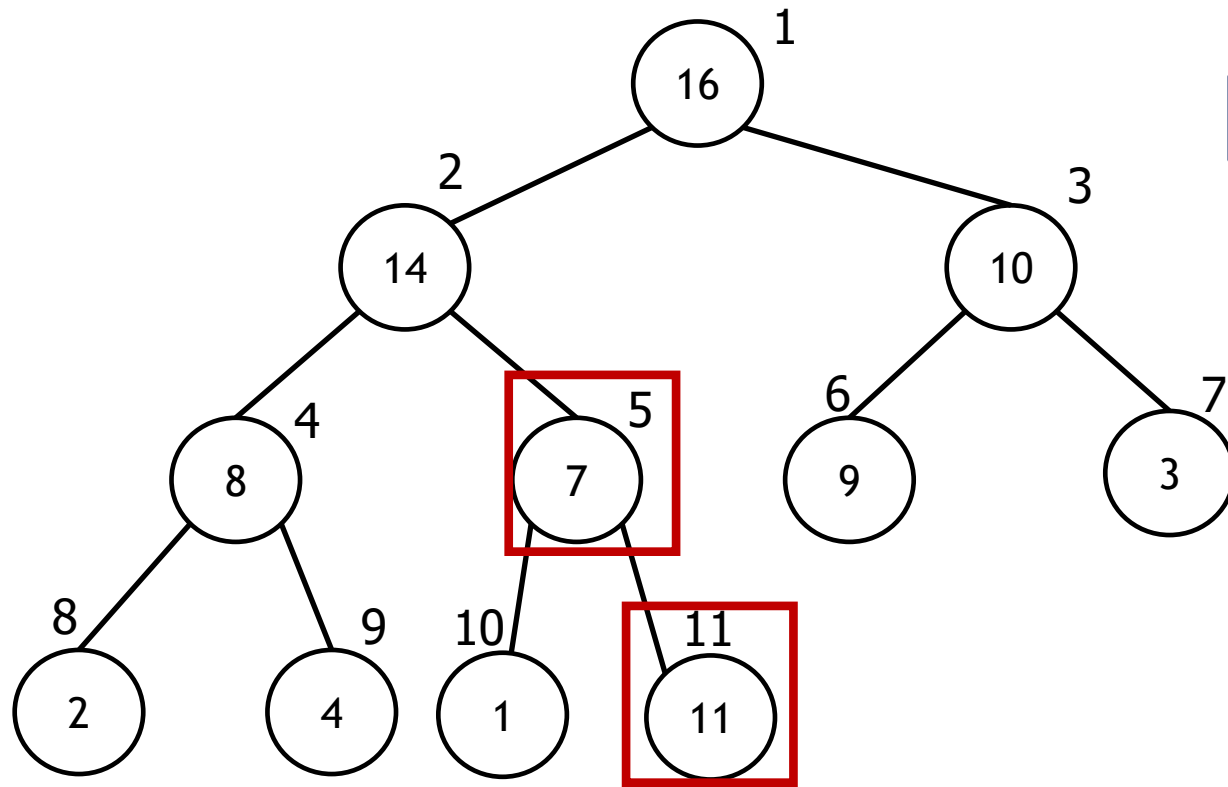
Add the element to the **bottom level** of the heap at the **leftmost open space**.



1	2	3	4	5	6	7	8	9	10	11
16	14	10	8	7	9	3	2	4	1	11

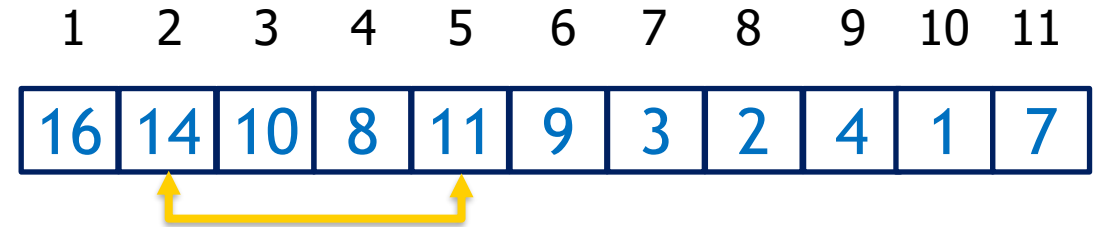
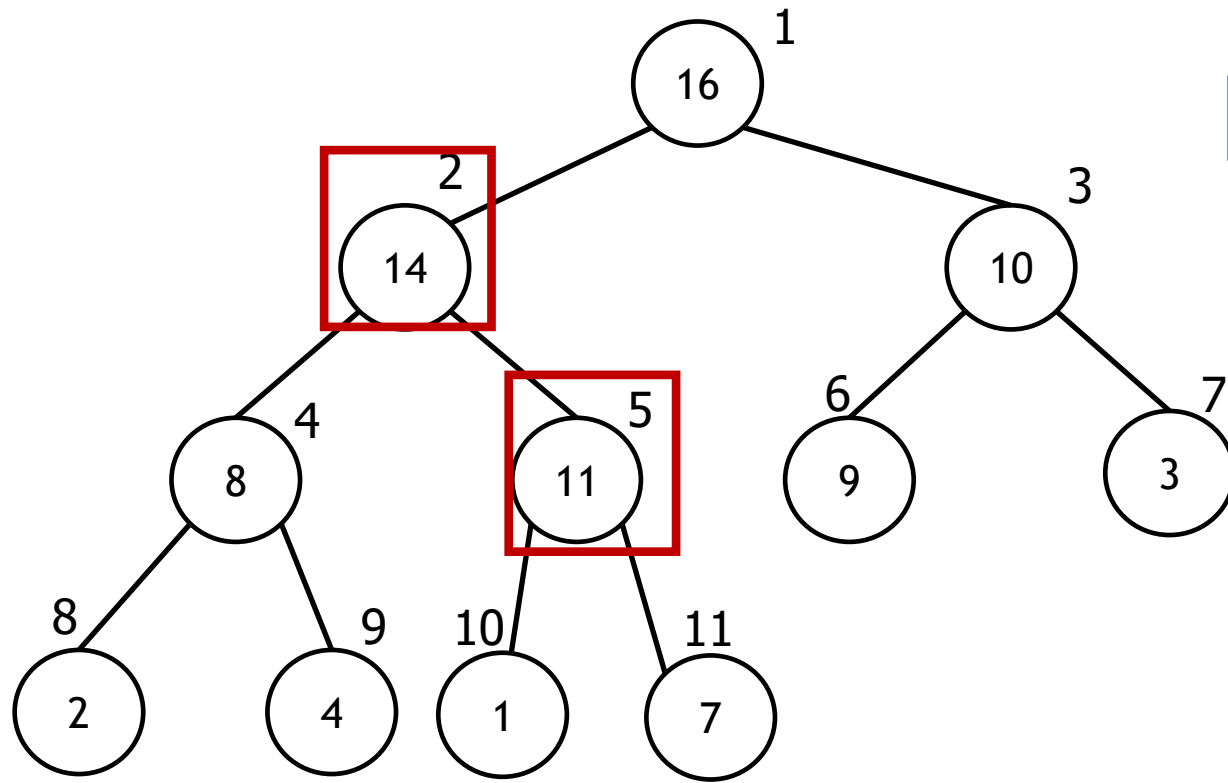
Heap Insertion

Compare the **added element** with its **parent**



Heap Insertion

Compare the **added element** with its **parent**



Correct order, Stop

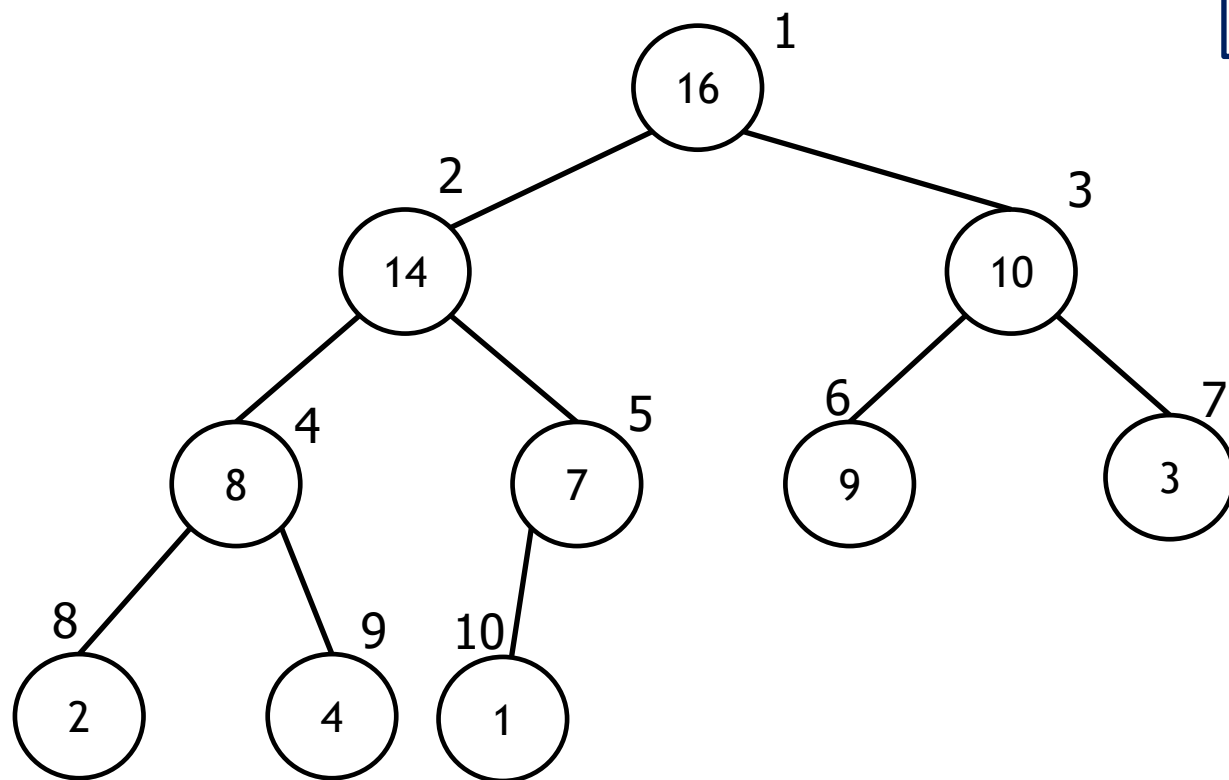
Heap Deletion

- To delete an element from a heap:
 - 1) Find the index i of the element we want to delete
 - 2) Swap this element with the last element
 - 3) $\text{Max-Heapify}(A, i)$

Heap Deletion

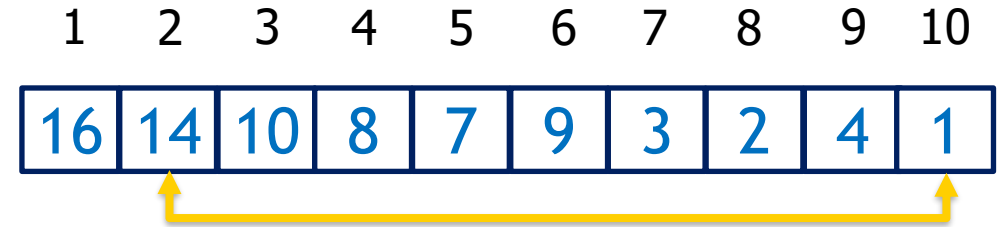
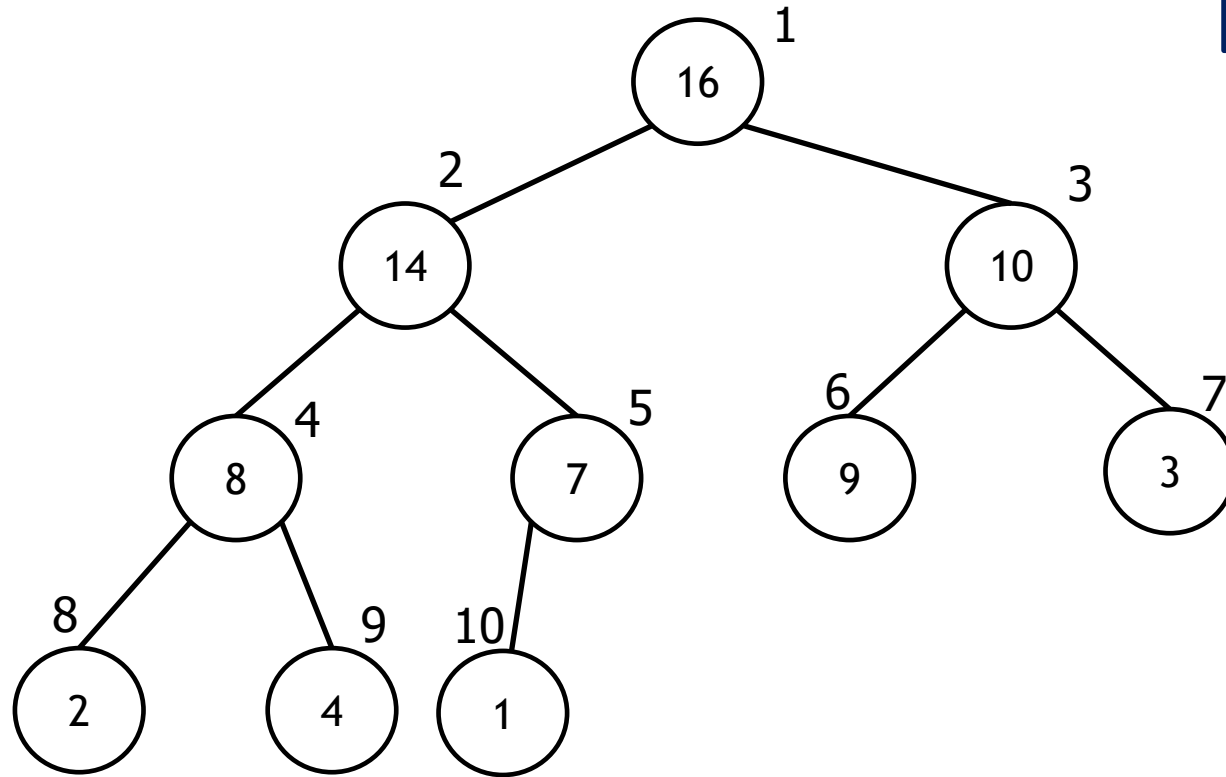
- Delete 14

1	2	3	4	5	6	7	8	9	10
16	14	10	8	7	9	3	2	4	1



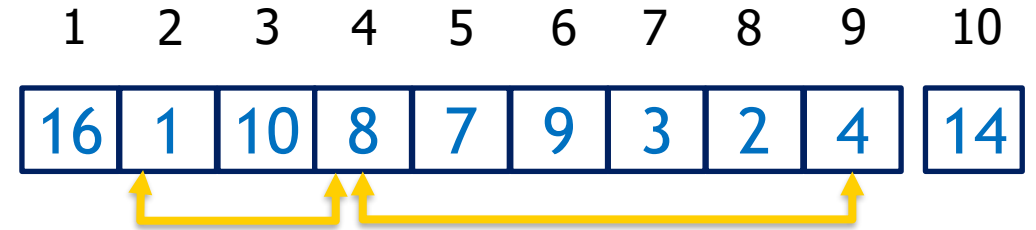
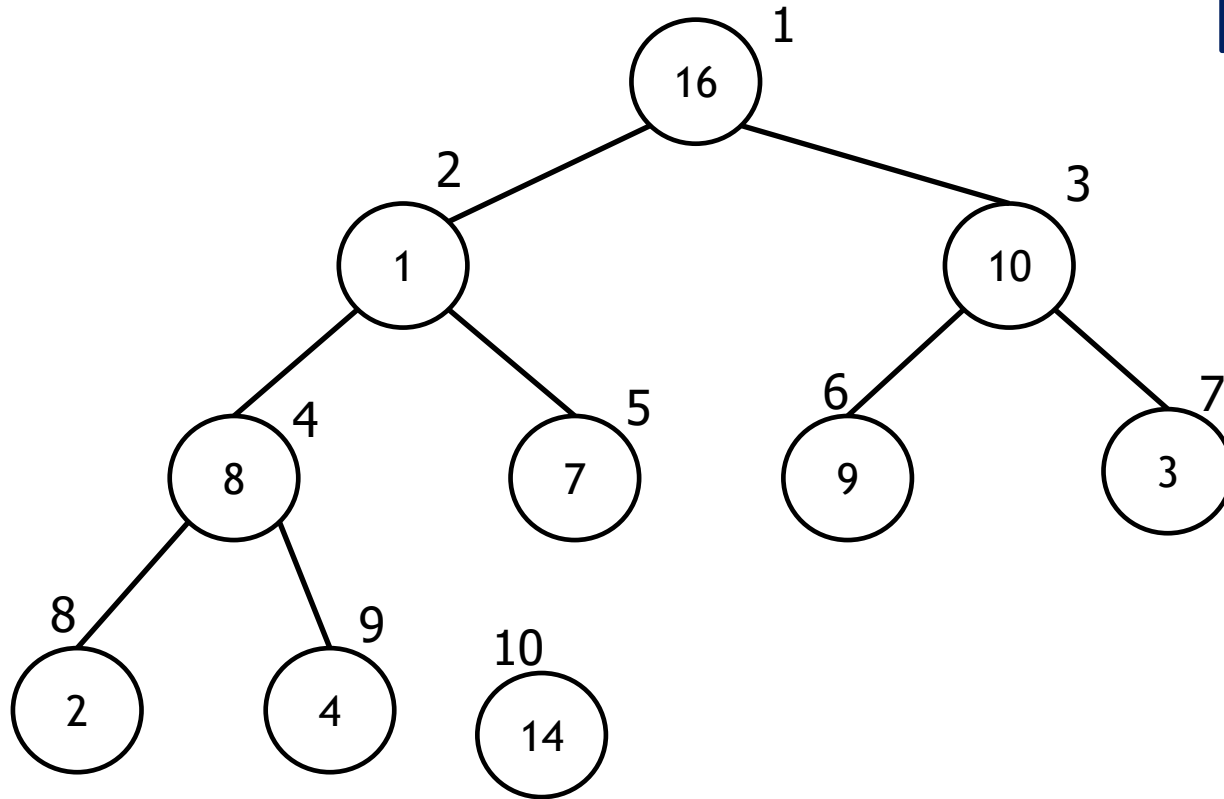
Heap Deletion

- Swap 14 with the last element



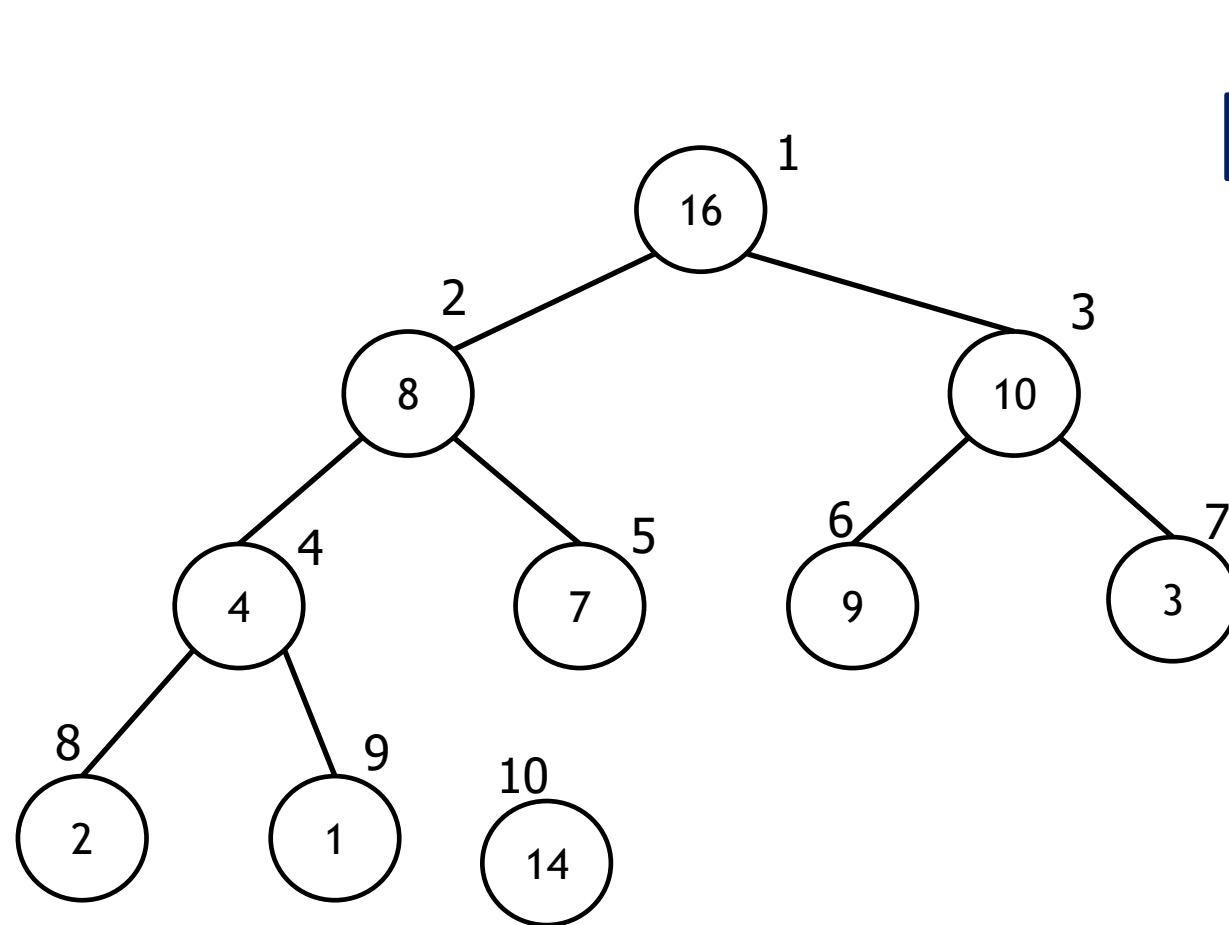
Heap Deletion

- $\text{MAX-heapify}(A, 2)$



Swap $A[2]$ and $A[4]$
Swap $A[4]$ and $A[9]$

Heap Deletion



1	2	3	4	5	6	7	8	9	10
16	8	10	4	7	9	3	2	1	14

Heapsort

- To sort an array using Heapsort:
 - 1) Build Max Heap
 - 2) Delete the root iteratively

Heapsort(A)

Build_Max_Heap(A)

for i = A.length downto 2

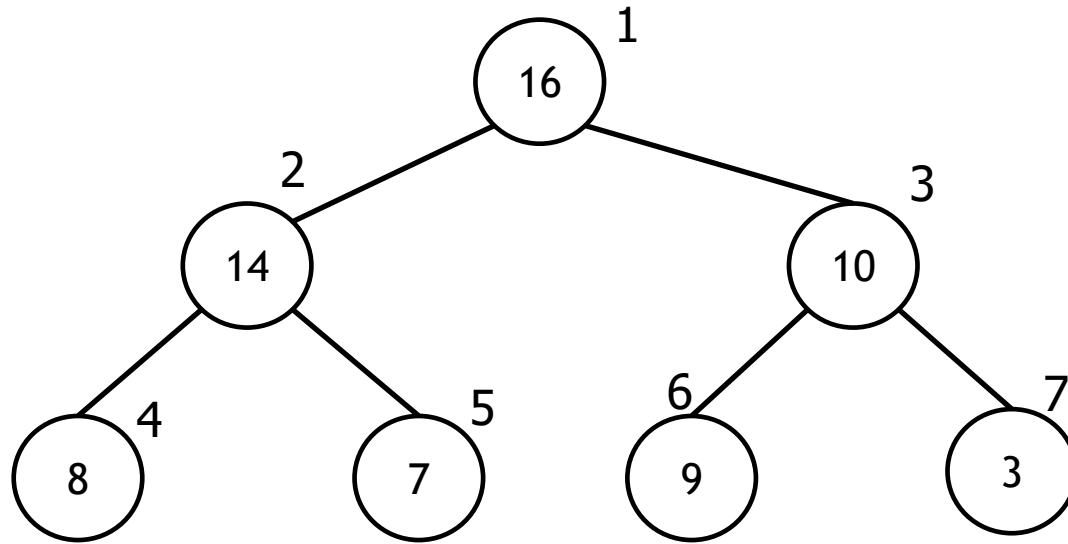
 exchange A[1] with A[i]

 A.heap-size = A.heap-size - 1

 Max_Heapify(A,1)

Heapsort

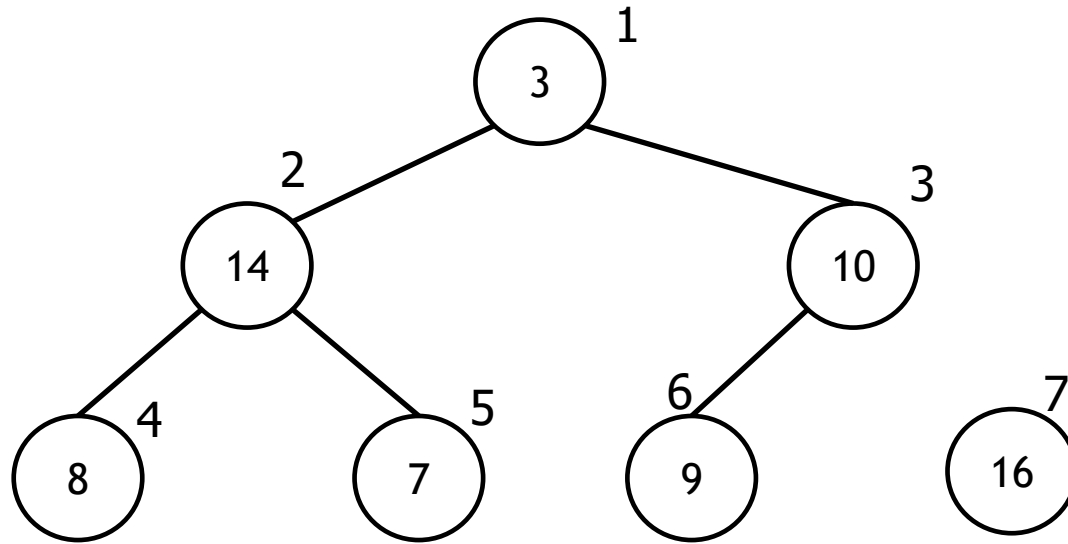
- Delete 16



1	2	3	4	5	6	7
16	14	10	8	7	9	3

Heapsort

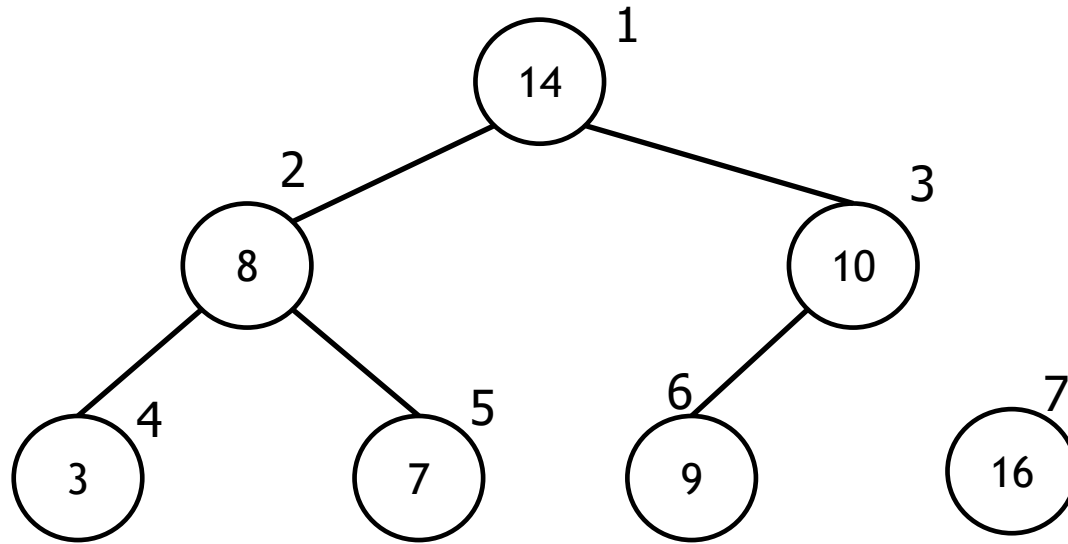
- Delete 16



1	2	3	4	5	6	7
3	14	10	8	7	9	16

Heapsort

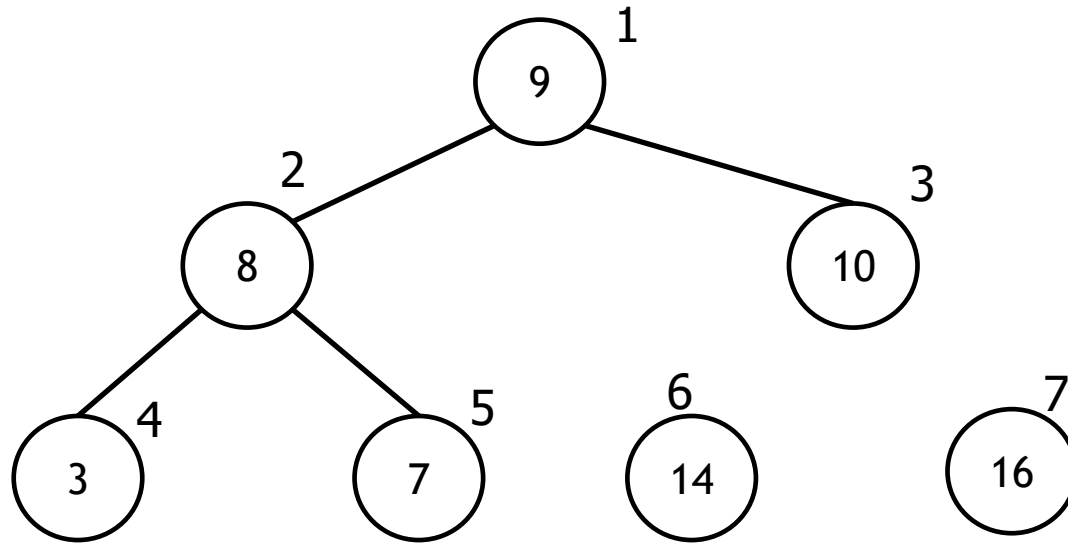
- Heapify(A,1)



1	2	3	4	5	6	7
14	8	10	3	7	9	16

Heapsort

- Delete 14

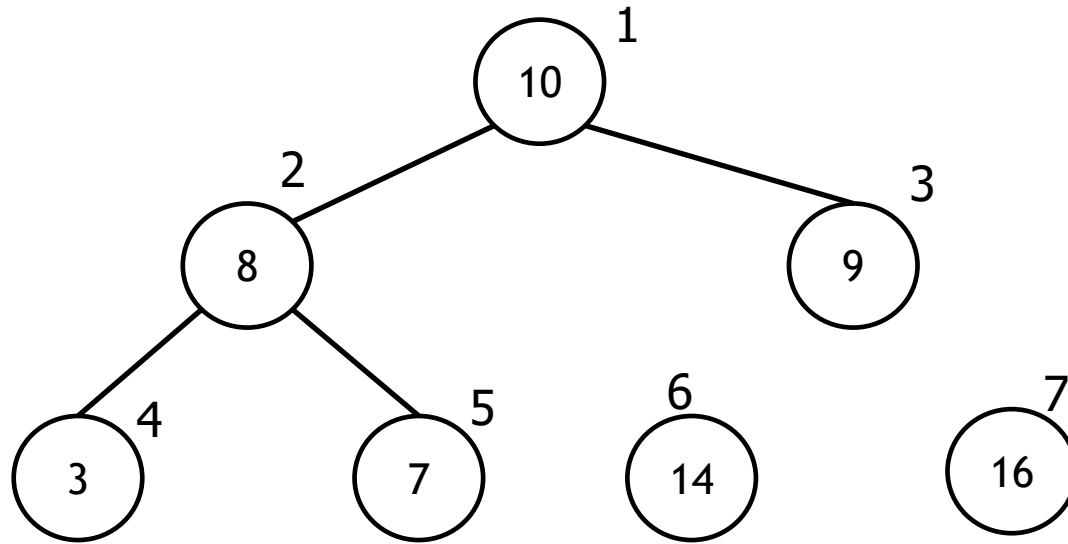


1	2	3	4	5
9	8	10	3	7

6	7
14	16

Heapsort

- $\text{Heapify}(A,1)$

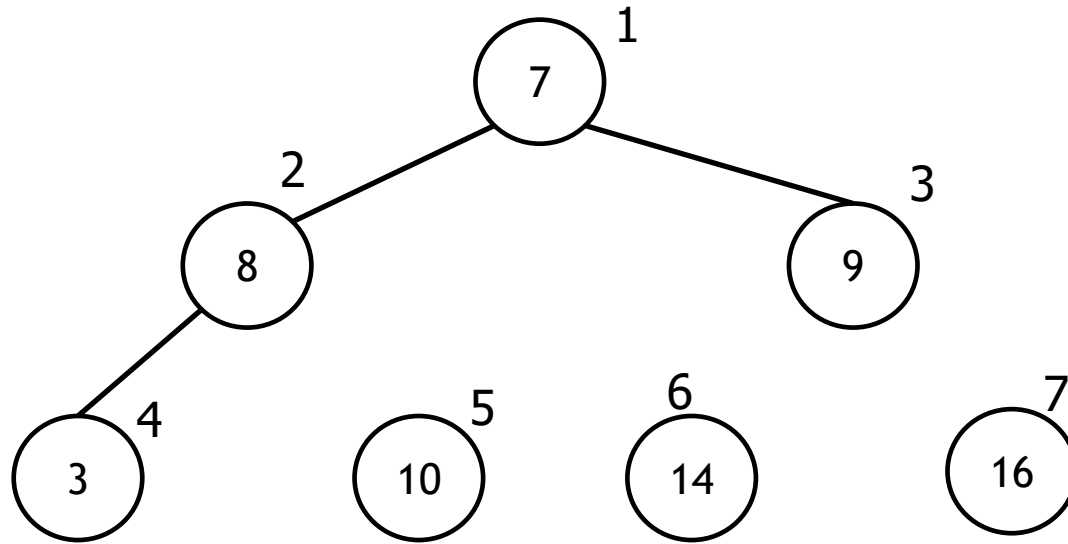


1	2	3	4	5
10	8	9	3	7

6	7
14	16

Heapsort

- Delete 10

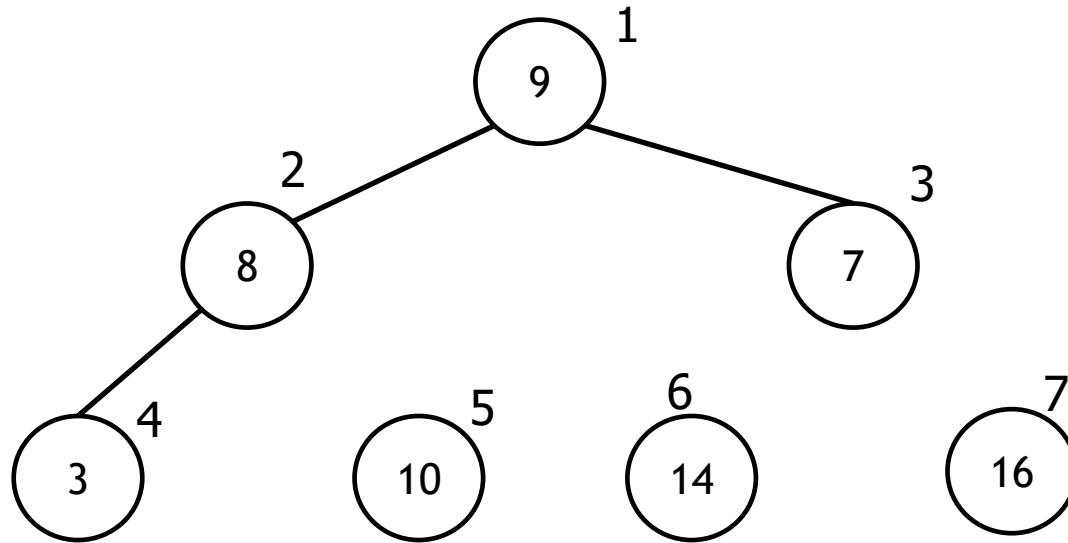


1	2	3	4
7	8	9	3

5	6	7
10	14	16

Heapsort

- Heapify(A,1)

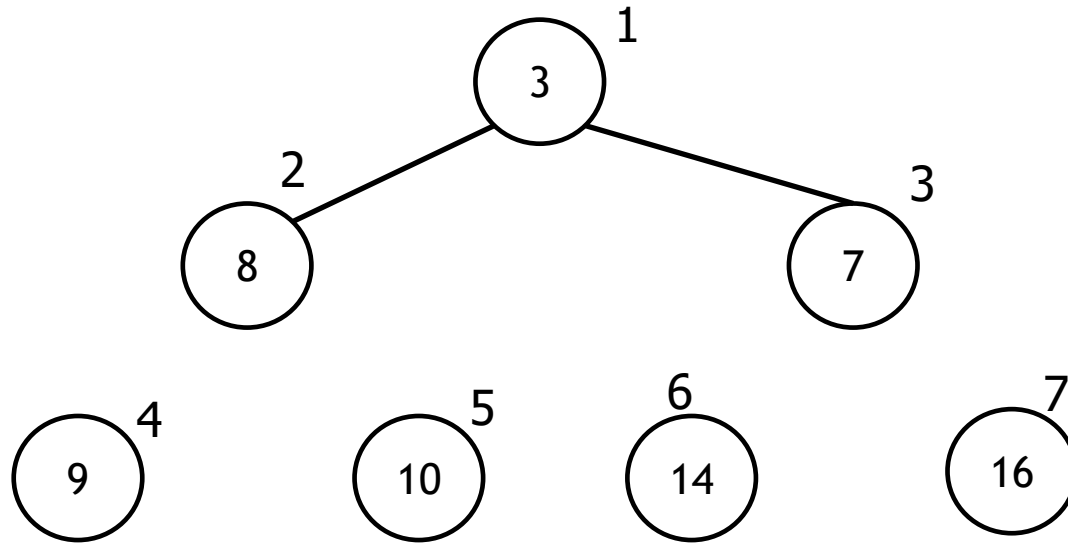


1	2	3	4
9	8	7	3

5	6	7
10	14	16

Heapsort

■ Delete 9

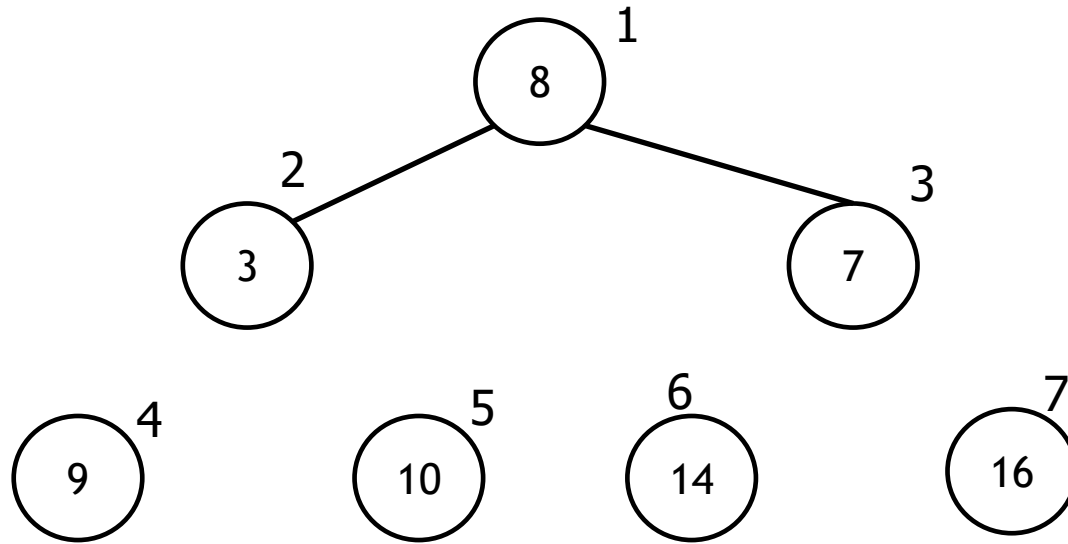


1	2	3
3	8	7

4	5	6	7
9	10	14	16

Heapsort

■ Heapify(A,1)

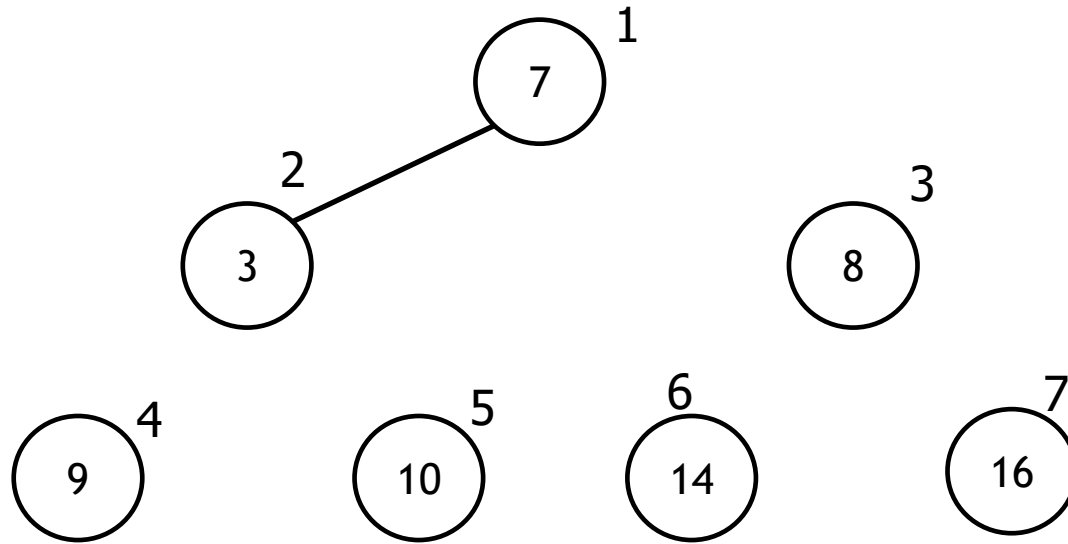


1	2	3
8	3	7

4	5	6	7
9	10	14	16

Heapsort

■ Delete 8

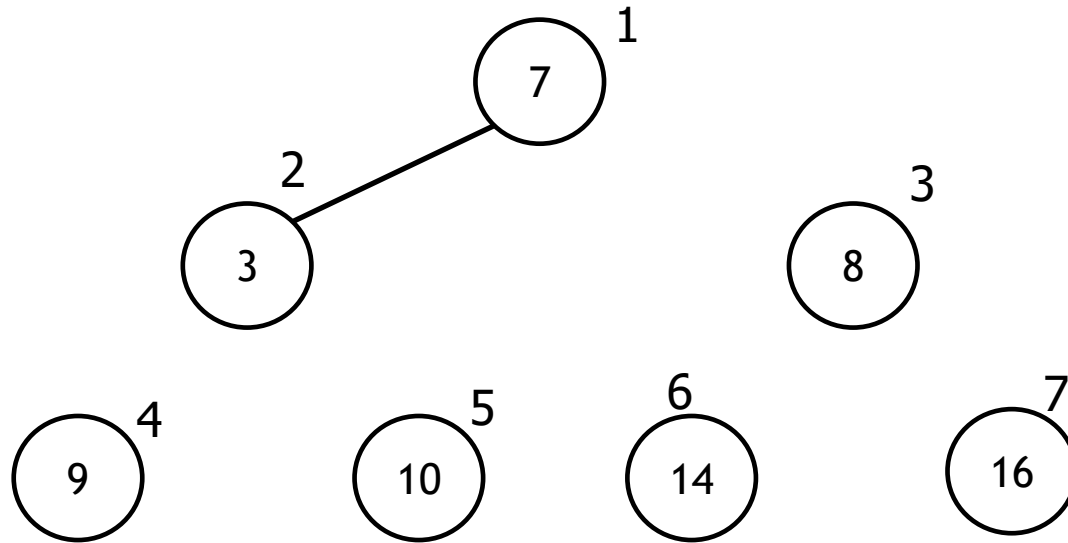


1	2
7	3

3	4	5	6	7
8	9	10	14	16

Heapsort

- $\text{Heapify}(A,1)$

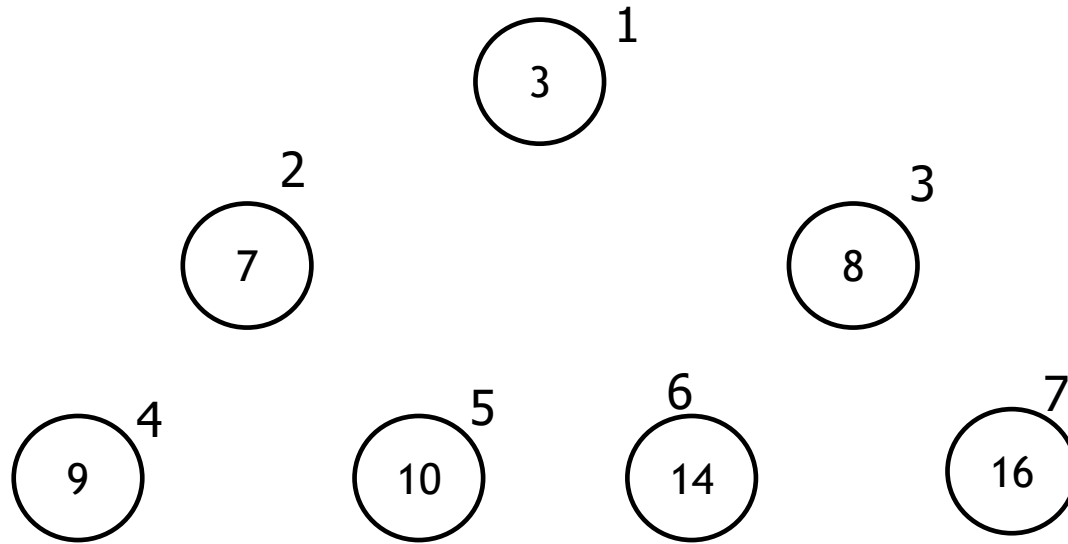


1	2
7	3

3	4	5	6	7
8	9	10	14	16

Heapsort

■ Delete 7

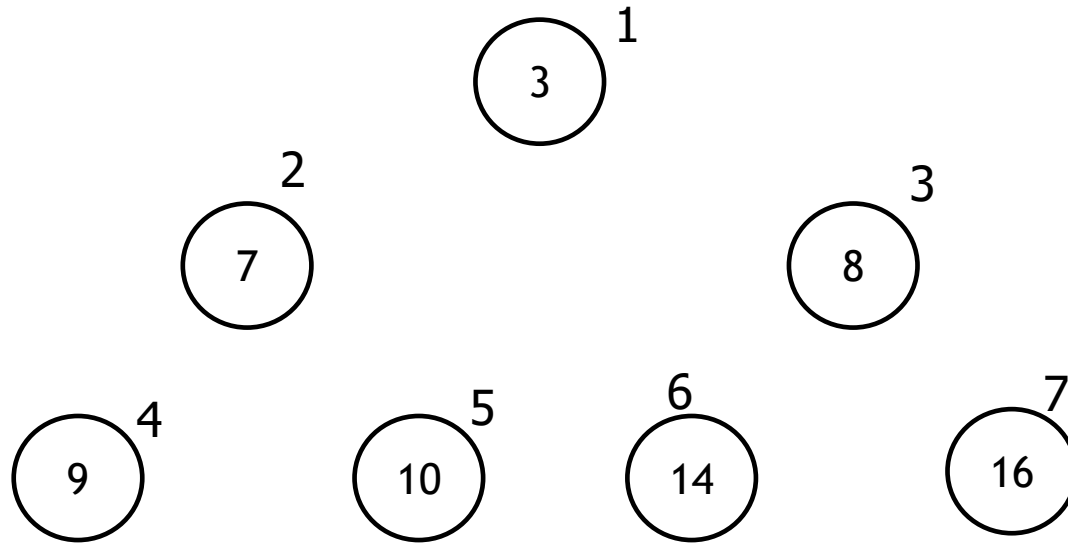


1
3

2	3	4	5	6	7
7	8	9	10	14	16

Heapsort

- Finished



1	2	3	4	5	6	7
3	7	8	9	10	14	16

Heapsort

- Time Complexity?
 - n iterations
 - Heapify takes $O(\log n)$

$O(n \log n)$

Priority Queue

- A data structure implementing a set S of elements, each associated with a key, supporting the following operations:
 - $\text{insert}(S, x)$: insert element x into set S
 - $\text{max}(S)$: return element of S with largest key
 - $\text{extract_max}(S)$: return element of S with largest key and remove it from S
 - $\text{increase_key}(S, x, k)$: increase the value of element x 's key to new value k
- One of the most popular applications of a heap: an efficient priority queue.
 - Insertion time complexity? $O(\log n)$
 - Deletion time complexity? $O(\log n)$

Exercise

- Given an array $A = [10, 20, 15, 12, 40, 25, 18]$, show the resulting max heap after the operation of heapify, and draw the corresponding binary tree.
- Given a max heap $H = [40, 30, 15, 10, 20]$, draw the intermediate heaps while conducting the heap sort algorithm.

Learning outcomes

- Stack
- Queue
- LinkedList
 - Doubly Linked List
- Array
- Tree
 - Binary Tree
- Heap
 - Heapsort